PROGRAMAÇÃO WEB · 2026/27 · AULA LABORATORIAL

# Ficha 6
# API RESTful de backend

Uma API que entrega os dados da Frutaria a qualquer aplicação: web, Android, iOS ou Windows. Pedidos e respostas HTTP, JSON, DTOs e camadas bem separadas. No fim, a entrada na autenticação com tokens JWT.




**Depois: login e token JWT**

PLANO DA AULA

# Como lá vamos chegar

**1 — REST e recursos**

O que é uma API, o estilo REST, recursos e boas práticas nos endpoints.

**2 — HTTP: pedido e resposta**

Anatomia do pedido e da resposta, verbos, status codes e headers.

**3 — JSON e DTOs**

O paralelo objeto ↔ JSON, e porque não se expõem as entidades.

**4 — As camadas da API**

Controller, DTO, Mapper, Service, Repository, e os erros.

**5 — Do pedido à BD**

GET, POST, PUT e DELETE em diagramas de sequência, etapa a etapa.

**6 — Mãos à obra**

Etapas 1 a 3 da ficha, alínea a alínea, com o guia HTML.

**A — A seguir: autenticação**

Login, token JWT, chaves, validade, e onde a app guarda o token.

**Como usar os materiais:** os passos 1 a 5 são a teoria. O passo 6 é a parte prática (slides verdes): a ordem das alíneas, os ficheiros a criar ou analisar, o que explicar em cada uma e o ponto de controlo. O código completo de cada ficheiro está no guia HTML da ficha 6.

PONTO DE PARTIDA

# Os dados já existem, falta uma porta para as apps

### Já existe (fichas 3 a 5)

- A base de dados `FrutariaPWeb`: 4 categorias, 4 modos de entrega e 19 produtos (dados de exemplo).
- As entidades `Categoria`, `Produto`, `ModoEntrega`, `Morada` e `ApplicationUser`, com validações.
- A Gestão de Loja (Blazor): o backoffice, para o Admin e o Gestor.
- Os roles `Admin`, `Gestor` e `Cliente`.

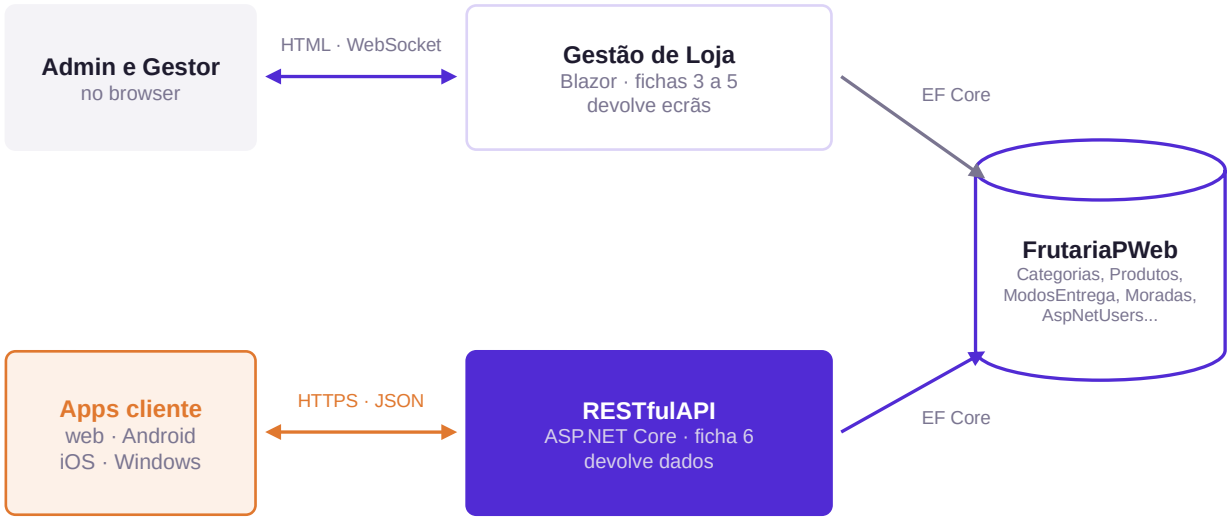
### Falta (esta ficha)

- Uma porta de entrada para as apps dos clientes: web, Android, iOS, Windows.
- Endereços (endpoints) que devolvem e recebem dados em JSON.
- Uma arquitetura em camadas: Controllers, DTOs, Mappers, Services, Repositories.
- Testar sem frontend: Swagger e Postman.
- Um endereço público, ainda em desenvolvimento: Dev Tunnel.

**A API é um projeto novo, RESTfulAPI, e não cria tabelas:** liga-se à mesma `FrutariaPWeb` e usa o mesmo modelo de dados da ficha 5, com o extra (a `Morada`). As migrations continuam a ser da Gestão de Loja.

CONTEXTO · ARQUITETURA

# Duas aplicações, uma base de dados




*As duas aplicações leem e gravam nas mesmas tabelas: um produto criado na Gestão de Loja aparece logo na API.*

### Gestão de Loja

O servidor desenha o ecrã. O browser só mostra o HTML que recebe.

### API

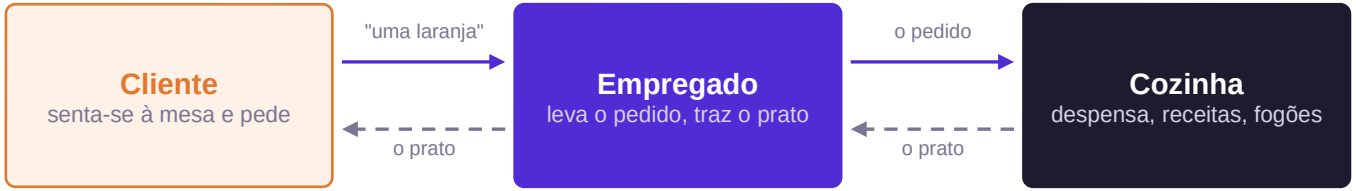
Devolve **só dados**. Quem desenha o ecrã é a app: cada uma à sua maneira.

### Porquê JSON

É texto. Qualquer linguagem o lê: C#, Kotlin, Swift, JavaScript.

CONCEITO · API

# Uma API é o empregado de mesa




*O cliente nunca entra na cozinha. Não sabe onde estão as panelas, nem precisa.*

| No restaurante | Na nossa API | Exemplo |
|---|---|---|
| o cliente | a app frontend | a app Android da Frutaria |
| a ementa | o contrato: a lista de endpoints | a página do Swagger |
| o pedido | o pedido HTTP | `GET /api/produtos/7` |
| o empregado | a API (os controllers) | `ProdutosController` |
| a cozinha | as regras e a base de dados | `ProdutoService`, `FrutariaPWeb` |
| o prato | a resposta, em JSON | `{ "nome": "Laranja", ... }` |
| "já acabou" | um status code de erro | `404 Not Found` |

### API

**A**pplication **P**rogramming **I**nterface: uma interface para programas, e não para pessoas.

### A vantagem

A cozinha pode mudar de fogão, ou de despensa, sem o cliente notar. A ementa (o contrato) é que não pode mudar.

CONTEXTO · FICHA 5 VS FICHA 6

# A Gestão de Loja manda ecrãs, a API manda dados

### Gestão de Loja (Blazor Server): o servidor desenha

O que chega ao browser: HTML pronto a mostrar

```
<tr>
  <td><img src="..." /></td>
  <td>Laranja</td>
  <td>1,49 €</td>
  <td><a href="/produtos/edit?id=1">Editar</a></td>
</tr>
```

**Ecrã feito no servidor**

- O aspeto (cores, botões, idioma) vem do servidor.
- Serve uma app: a que corre no browser.
- Mudar o aspeto obriga a mexer no servidor.

### RESTfulAPI: só os dados, a app desenha

GET /api/produtos/1 → o que chega à app

```
{
  "id": 1,
  "nome": "Laranja",
  "preco": 1.49,
  "imagem": "iVBORw0KGgo..."
}
```

**Dados, cada app desenha**

- `1.49` é um número: o Android mostra "1,49 €", uma app inglesa "€1.49".
- Serve **todas** as apps ao mesmo tempo.
- O aspeto muda na app; a API fica igual.

CONCEITO · REST

# REST: um conjunto de regras, não uma biblioteca

**REST** (REpresentational State Transfer) é um estilo de desenhar APIs sobre HTTP, descrito por Roy Fielding em 2000. Uma API que segue as regras diz-se **RESTful**.

### 1. Recursos com endereço

Cada coisa tem um URL: `/api/produtos/7` é o produto 7.

### 2. Interface uniforme

Os mesmos verbos HTTP para tudo: GET lê, POST cria, PUT altera, DELETE apaga.

### 3. Representações

O cliente não recebe o produto, recebe uma representação dele: JSON.

### 4. Sem estado (stateless)

Cada pedido leva tudo o que é preciso. O servidor não se lembra do anterior.

### 5. Cliente e servidor separados

A app e a API evoluem cada uma por si. Só o contrato as liga.

### 6. Respostas cacheáveis

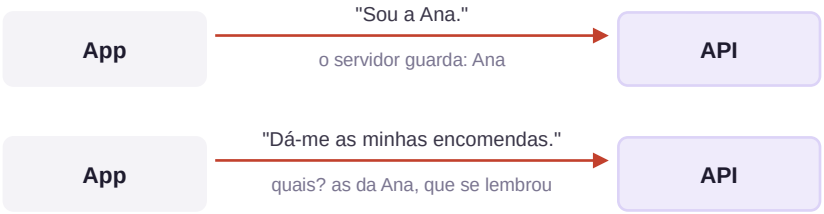
A resposta diz se pode ser guardada e reutilizada (os headers de cache).

**Em resumo, para esta ficha:** endereços que identificam coisas (recursos), verbos que dizem o que fazer, JSON como formato, e nenhum pedido depende do anterior. A regra 4 volta a ser essencial na autenticação: é por ela que se usa um token.

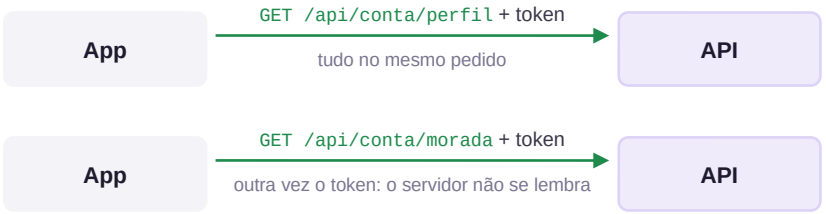
CONCEITO · SEM ESTADO

# Stateless: o servidor não se lembra de ti

## Com estado: como uma conversa




## Sem estado (REST): como uma carta




### O problema

Se o servidor reiniciar, ou se o pedido for parar a outro servidor, "a Ana" perdeu-se.

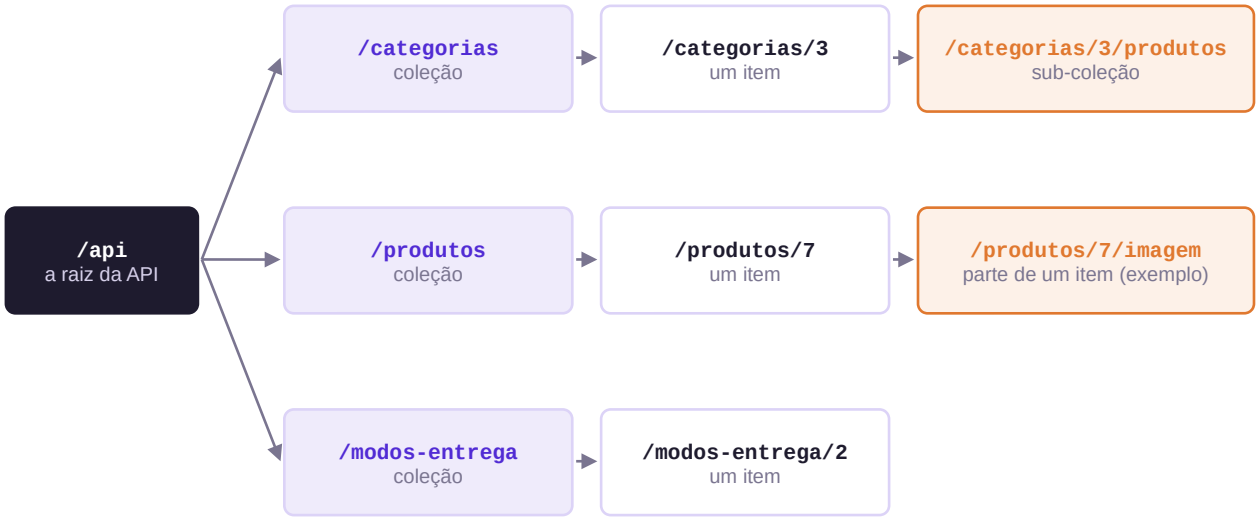
### A vantagem

Qualquer servidor responde a qualquer pedido. Reiniciar a API não "desliga" ninguém.

**Consequência prática:** na API, a app manda a identificação **em todos os pedidos**, no header `Authorization`. É isso o token, na última parte da aula.

CONCEITO · RECURSOS

# Recurso: uma coisa com endereço




*Lê-se da esquerda para a direita, como pastas: a categoria 3 está dentro das categorias; os produtos da categoria 3 estão dentro da categoria 3.*

### Coleção e item

`/produtos`: todos.
`/produtos/7`: só um, pelo `Id`.

### Recurso ≠ tabela

O produto 7 em JSON junta duas tabelas (a categoria vem com ele). E `/imagem` seria só uma coluna.

### Endereço + verbo

O URL diz **qual** coisa; o verbo HTTP diz **o que** lhe fazer.

BOAS PRÁTICAS · ENDPOINTS

# Desenhar endpoints: o certo e o errado

| Evitar | Preferir | A regra |
|---|---|---|
| `GET /api/getProdutos` | `GET /api/produtos` | Substantivos, sem verbos: o verbo é o método HTTP |
| `GET /api/produto/7` | `GET /api/produtos/7` | Plural, mesmo para um item: é o item 7 da coleção |
| `GET /api/produtos/apagar?id=7` | `DELETE /api/produtos/7` | O id vai no caminho; a ação vai no verbo |
| `POST /api/produtos/7/update` | `PUT /api/produtos/7` | Não inventar verbos no URL |
| `GET /api/ModosEntrega` | `GET /api/modos-entrega` | Minúsculas; palavras separadas por hífen |
| `GET /api/produtosDaCategoria?c=3` | `GET /api/categorias/3/produtos` | Relações como hierarquia |
| `GET /api/produtos/promocoes` | `GET /api/produtos?promocao=true` | Filtros, pesquisa e ordenação na query string |
| `GET /api/produtos.json` | `Accept: application/json` | O formato vai nos headers, não no URL |
| `200 OK` com `{"erro": "..."}` | `404 Not Found` | O status code diz se correu bem; não esconder erros num 200 |

**A exceção:** ações que não são criar, ler, alterar ou apagar uma coisa. O login é a mais comum: `POST /api/auth/login`.

A NOSSA API · O CONTRATO

# Os endpoints da RESTfulAPI

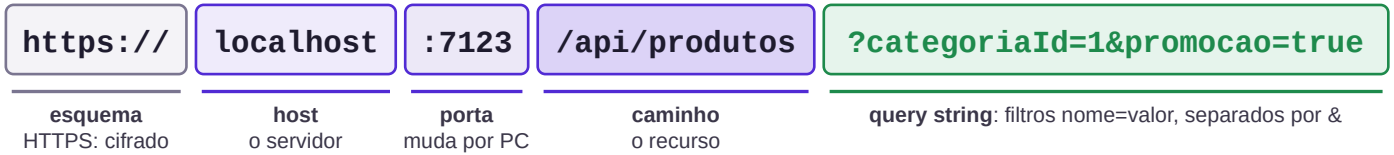
| Verbo | Endpoint | O que faz | Sucesso | Erros |
|---|---|---|---|---|
| GET | `/api/categorias` | Lista, ordenada pela `Ordem` | 200 | · |
| GET | `/api/categorias/{id}` | Uma categoria | 200 | 404 |
| GET | `/api/categorias/{id}/produtos` | Os produtos da categoria | 200 | 404 |
| POST | `/api/categorias` | Cria | 201 | 400, 409 |
| PUT | `/api/categorias/{id}` | Altera | 204 | 400, 404, 409 |
| DELETE | `/api/categorias/{id}` | Apaga, se não tiver produtos | 204 | 404, 409 |
| GET | `/api/produtos?categoriaId=1&promocao=true` | Lista, com filtros opcionais | 200 | · |
| GET | `/api/produtos/{id}` | Um produto, com categoria e modo | 200 | 404 |
| POST PUT DELETE | `/api/produtos` e `/api/produtos/{id}` | Como nas categorias | 201 · 204 | 400, 404, 409 |
| Os quatro | `/api/modos-entrega` e `/{id}` | O mesmo CRUD | 200 · 201 · 204 | 400, 404, 409 |

*Os status codes explicam-se daqui a pouco. `{id}` é um parâmetro de rota: no pedido real, um número (`/api/categorias/3`).*

HTTP · O ENDEREÇO

# Anatomia de um URL




| Parte | Exemplo | No controller (C#) |
|---|---|---|
| caminho | `/api/produtos` | `[Route("api/produtos")]` |
| parâmetro de rota | `/7` | `[HttpGet("{id:int}")]` → `int id` |
| query string | `?categoriaId=1` | `[FromQuery] int? categoriaId` |

### A porta e o host mudam

A porta vem do `launchSettings.json`. Com o Dev Tunnel muda o host todo.

### Rota ou query?

A **rota** identifica; a **query** refina, e é opcional.

HTTP · O PEDIDO

# O pedido HTTP: um envelope com quatro partes

POST /api/produtos: criar um produto (texto que viaja na rede)

```
POST /api/produtos HTTP/1.1
Host: localhost:7123
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Length: 168

{
  "nome": "Manga",
  "detalhe": "Manga madura, ao quilo",
  "preco": 3.49,
  "emStock": 20,
  "disponivel": true,
  "categoriaId": 1,
  "modoEntregaId": 1
}
```

1

2

3

4

### Linha de pedido

O **verbo** (o que fazer), o **caminho** (a quem) e a versão do HTTP.

### Headers

Pares `Nome: valor` com informação sobre o pedido: o formato do body, o formato que se aceita na resposta, quem sou (o token).

### Linha em branco

Separa os headers do body. É obrigatória.

### Body (corpo)

Os dados, em JSON. Só em **POST**, **PUT** e **PATCH**: um GET ou um DELETE não leva body.

É tudo **texto**; o HTTPS cifra-o durante a viagem.

HTTP · A RESPOSTA

# A resposta HTTP: status, headers e body

A resposta da API ao POST anterior

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: https://localhost:7123/api/produtos/20
Date: Tue, 03 Nov 2026 12:00:00 GMT

{
  "id": 20,
  "nome": "Manga",
  "preco": 3.49,
  "categoria": { "id": 1, "nome": "Fruta Fresca" },
  "imagem": "iVBORw0KGgoAAAANSUhEUgAA..."
}
```

### 1 · Linha de estado

A versão, o **status code** (`201`) e o seu nome (`Created`). É a primeira coisa que a app lê.

### 2 · Headers

`Location`: onde ficou o recurso novo. `Content-Type`: o body é JSON em UTF-8 (os acentos de "Maçã").

### 3 · Body

O produto criado, já com o `id` que a base de dados lhe deu. Num `204 No Content`, não há body.

**Pedido e resposta andam sempre aos pares:** a app pergunta, a API responde, e a ligação fica livre para o pedido seguinte.

HTTP · VERBOS

# Os verbos: o que fazer ao recurso

| Verbo | Para quê | Exemplo | Body no pedido | Resposta típica |
|---|---|---|---|---|
| GET | Ler | `GET /api/produtos/7` | não | `200 OK` + JSON |
| POST | Criar | `POST /api/produtos` | sim: o novo | `201 Created` + `Location` + JSON |
| PUT | Alterar (tudo) | `PUT /api/produtos/7` | sim: completo | `204 No Content` |
| PATCH | Alterar (uma parte) | `PATCH /api/produtos/7` | sim: só os campos | `204` ou `200` (não usamos) |
| DELETE | Apagar | `DELETE /api/produtos/7` | não | `204 No Content` |

### PUT ou PATCH: mudar só o preço

PUT: o produto inteiro

```
{ "nome": "Laranja",
  "detalhe": "...",
  "preco": 1.59,  ...todos }
```

PATCH: só o que muda

```
{ "preco": 1.59 }
```

### Cuidado com o PUT

O PUT **substitui**: um campo que não venha fica com o valor por omissão (`null`, `0`, `false`). A app manda sempre o objeto todo, mesmo que só tenha mudado o preço.

LIGAÇÃO À FICHA 5 · CRUD

# CRUD, HTTP, SQL e EF Core: a mesma ideia

| Operação | Verbo HTTP | SQL (o que a BD recebe) | EF Core (o que já usaram) |
|---|---|---|---|
| **Create** Criar | POST | `INSERT INTO Produtos ...` | `Add(p)` + `SaveChangesAsync()` |
| **Read** Ler | GET | `SELECT ... FROM Produtos` | `ToListAsync()`, `FirstOrDefaultAsync()` |
| **Update** Alterar | PUT | `UPDATE Produtos SET ...` | alterar o objeto + `SaveChangesAsync()` |
| **Delete** Apagar | DELETE | `DELETE FROM Produtos ...` | `Remove(p)` + `SaveChangesAsync()` |

**Na ficha 5** o clique em Gravar chamava diretamente o EF Core. **Na API**, o clique acontece noutro programa (a app): chega como um pedido HTTP, e é a API que o traduz, camada a camada, até ao SQL.

HTTP · REPETIR PEDIDOS

# E se o pedido for enviado duas vezes?

| Verbo | Seguro? | Idempotente? | Se for repetido... |
|---|---|---|---|
| GET | sim | sim | Nada muda. Pode repetir-se à vontade. |
| PUT | não | sim | O produto fica igual à segunda vez. |
| DELETE | não | sim | 1.ª vez `204`, 2.ª vez `404`: mas o produto continua apagado. |
| POST | não | não | **Dois produtos!** (ou `409`, se o nome tiver de ser único). |

### Seguro

Não altera nada no servidor. Só lê.

### Idempotente

Fazer uma vez ou dez vezes deixa o servidor **no mesmo estado**.

### O elevador

Carregar cinco vezes no botão do 3.º andar é o mesmo que carregar uma: **idempotente**.

### A máquina de café

Carregar duas vezes dá dois cafés: **não idempotente**. Num telemóvel com rede fraca, a app pode repetir um GET ou um PUT; um POST, não.

HTTP · STATUS CODES

# O status code: a resposta num número

## 1xx

**Informação**

*"Recebido, continua."*

Quase nunca se vê.

## 2xx

**Sucesso**

*"Correu bem."*

200, 201, 204

## 3xx

**Redirecionamento**

*"Está noutro sítio."*

301, 307

## 4xx

**Erro do cliente**

*"O pedido está mal."*

400, 401, 403, 404, 409

## 5xx

**Erro do servidor**

*"A culpa é nossa."*

500, 503

### 4xx: o cliente pode corrigir

Mudar o pedido e tentar outra vez: preencher o preço, fazer login, escolher outro nome.

### 5xx: não adianta repetir igual

É um erro no servidor (uma exceção, a BD desligada). Vê-se na consola da API, não na app.

HTTP · STATUS CODES

# Os status codes da nossa API

| | Nome | Quando | Exemplo na RESTfulAPI |
|---|---|---|---|
| 200 | OK | Correu bem e há dados para devolver | `GET /api/produtos/7` encontrou o produto |
| 201 | Created | Criou-se um recurso novo | `POST /api/produtos`; `Location` diz onde |
| 204 | No Content | Correu bem, sem nada para devolver | `PUT` ou `DELETE` com sucesso |
| 400 | Bad Request | Os dados estão mal | preço `0`; categoria que não existe |
| 401 | Unauthorized | Não se sabe quem és (sem token, ou inválido) | próxima ficha: sem `Authorization` |
| 403 | Forbidden | Sabe-se quem és, mas não podes | próxima ficha: um Cliente a apagar |
| 404 | Not Found | O recurso não existe | `GET /api/produtos/999` |
| 409 | Conflict | Choca com o estado atual dos dados | nome repetido; categoria com produtos |
| 415 | Unsupported Media Type | O body não vem no formato esperado | faltou `Content-Type: application/json` |
| 500 | Internal Server Error | Exceção no servidor | a connection string está errada |

*O 401 chama-se "Unauthorized" por razões históricas, mas quer dizer "não autenticado". O 403 é que é "sem autorização".*

HTTP · HEADERS

# Headers: a informação sobre o pedido

| Header | Vai no... | Para quê | Exemplo |
|---|---|---|---|
| `Content-Type` | pedido e resposta | O formato do body | `application/json; charset=utf-8` |
| `Accept` | pedido | O formato que a app quer receber | `application/json` |
| `Authorization` | pedido | Quem sou: o token | `Bearer eyJhbGciOi...` |
| `Location` | resposta (201) | O endereço do recurso criado | `/api/produtos/20` |
| `WWW-Authenticate` | resposta (401) | Como se autenticar, e porque falhou | `Bearer error="invalid_token"` |
| `Access-Control-Allow-Origin` | resposta | CORS: que sites podem chamar a API | `https://frutaria.pt` |
| `X-Tunnel-Skip-AntiPhishing-Page` | pedido | Saltar o aviso do Dev Tunnel | `true` |

**Nomes sem maiúsculas obrigatórias:** `content-type` e `Content-Type` são o mesmo header.

**Os dois que mais vão escrever:** `Content-Type` no Postman e `Authorization` na app.

JSON · FORMATO

# JSON: texto com seis tipos de valores

Um produto em JSON

```
{
  "id": 1,
  "nome": "Laranja",
  "preco": 1.49,
  "promocao": true,
  "origem": null,
  "categoria": {
    "id": 1,
    "nome": "Fruta Fresca"
  },
  "etiquetas": ["Algarve", "biológico"]
}
```

| Tipo | Escreve-se | No exemplo |
|---|---|---|
| objeto | `{ "chave": valor }` | o produto, a categoria |
| array (lista) | `[ v1, v2 ]` | `etiquetas` |
| string | `"entre aspas"` | `"Laranja"` |
| número | `1.49` | `id`, `preco` |
| booleano | `true` / `false` | `promocao` |
| nulo | `null` | `origem` |

## Regras que dão erro

- Chaves e strings com **aspas duplas** (`'Laranja'` não vale).
- Sem vírgula depois do último elemento.
- Sem comentários. Números com **ponto**: `1.49`, não `1,49`.

JSON · O PARALELO

# Um objeto C# e o seu JSON, lado a lado

C#: um objeto ProdutoDto em memória

```
var dto = new ProdutoDto
{
    Id = 1,
    Nome = "Laranja",
    Preco = 1.49m,
    Promocao = true,
    Origem = null,
    Categoria = new ReferenciaDto
        { Id = 1, Nome = "Fruta Fresca" },
    Etiquetas = ["Algarve", "biológico"]
};
```

serializar →

← desserializar

JSON: o mesmo objeto, em texto

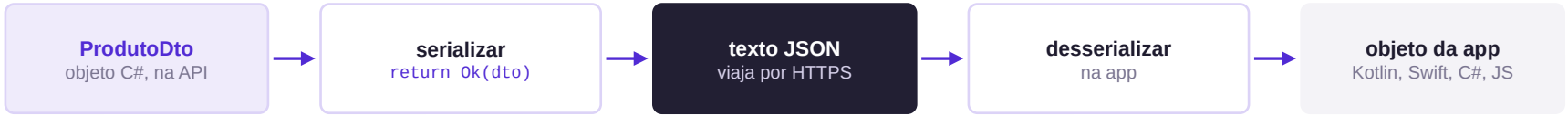
```
{
  "id": 1,
  "nome": "Laranja",
  "preco": 1.49,
  "promocao": true,
  "origem": null,
  "categoria":
    { "id": 1, "nome": "Fruta Fresca" },
  "etiquetas": ["Algarve", "biológico"]
}
```

| C# | JSON | Repare |
|---|---|---|
| propriedade `Preco` | chave `"preco"` | **PascalCase** no C#, **camelCase** no JSON: o ASP.NET Core converte |
| objeto dentro de objeto | `{ }` dentro de `{ }` | a categoria vem **aninhada** no produto |

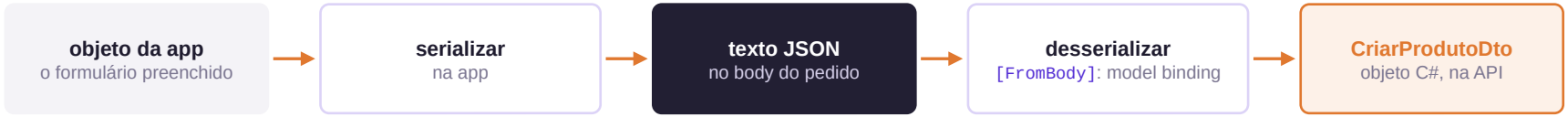
JSON · SERIALIZAÇÃO

# Quem converte, e quando

## GET: da API para a app




## POST e PUT: da app para a API




### Ninguém escreve JSON à mão

O ASP.NET Core usa o `System.Text.Json`, sozinho, à saída e à entrada.

### Campos a mais

Uma chave no JSON que o DTO não tem é **ignorada**. É isto que trava o ataque do slide dos DTOs.

### Campos em falta

Uma chave que não vem fica com o **valor por omissão**: `0`, `null`, `false`. Por isso se valida.

JSON · TIPOS

# Tipos C# em JSON: e os bytes da imagem?

| C# | JSON | Exemplo |
|---|---|---|
| `int`, `long`, `decimal` | número | `20`, `1.49` |
| `string` | string | `"Laranja"` |
| `bool` | booleano | `true` |
| `string?` vazio, `int?` vazio | `null` | `"origem": null` |
| `DateTime` | string ISO 8601 | `"2026-11-03T12:00:00Z"` |
| `List<T>`, `T[]` | array | `[1, 2, 3]` |
| uma classe | objeto | `{ "id": 1, ... }` |
| `byte[]` | string **Base64** | `"iVBORw0KGgoAAAANSUhEUgAA..."` |

## A imagem vira texto

Um `byte[]` vai como Base64: cada 3 bytes dão 4 letras, **33% maior**. Uma imagem de 10 KB dá 13 KB de texto, dentro do JSON.

## A nossa decisão (não a ideal)

Os DTOs levam a imagem em Base64 (`imagem`), a entrar (POST, PUT) e a sair (GET): um pedido só, e as listagens já com as imagens. **Não é a forma mais correta**: a correta é a imagem ter um endpoint seu (`/api/produtos/{id}/imagem`), e o JSON levar só o endereço.

JSON · O PROBLEMA

# Devolver a entidade tal como está: o que corre mal

```
return Ok(produto)  // a entidade Produto, sem DTO
{
  "id": 7,
  "nome": "Uvas",
  "imagem": "iVBORw0KGgoAAAANSUhEUgAAAPAAAAB4CAYAAAA6
             ...mais 13 000 caracteres...",
  "categoriaId": 1,
  "categoria": null,
  "modoentrega": null,
  ...
}
// e com ApplicationUser:
{ "email": "admin@localhost.com",
  "passwordHash": "AQAAAAIAAYagAAAAEJk...",
  "securityStamp": "QX7D...", "nif": 123456789 }
```

### 1 · Fuga de dados

O hash da palavra-passe, o NIF, o estado da conta: tudo o que a tabela tem, sai.

### 2 · Peso a mais

Tudo o que a tabela tem, quer a app precise ou não. Com um DTO, escolhe-se o que vai.

### 3 · Dados a meio

Sem `Include`, a `categoria` vem `null`, e a app não sabe se é erro ou se não tem.

### 4 · Ciclos

Com `Categoria.Produtos` e `Produto.categoria`, o JSON não acaba: `JsonException: A possible object cycle was detected`.

CAMADAS · A IDEIA

# Cada um no seu posto: a brigada da cozinha




**O prato empratado**
o que sai da cozinha para a mesa

**DTO, e o empratar é o Mapper**

Os ingredientes crus (as **entidades**) ficam na cozinha: nunca vão para a mesa.

### Um posto, uma tarefa

O empregado não cozinha. O chefe não vai à despensa. Cada um faz só o seu.

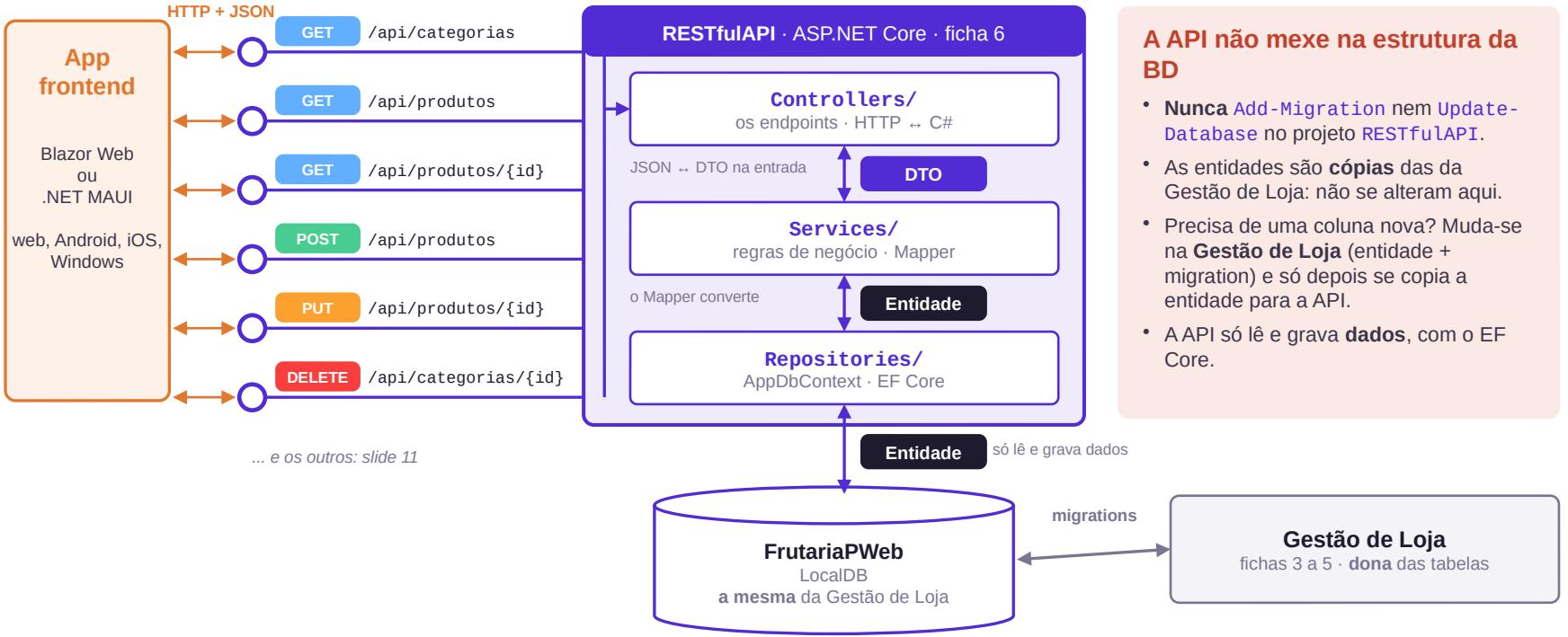
### Trocar sem partir

Mudar de despensa (outra BD) só obriga a mudar o despenseiro.

### Encontrar o erro

Um preço mal calculado? É no chefe. Um JOIN em falta? No despenseiro.

# A RESTfulAPI de ponta a ponta

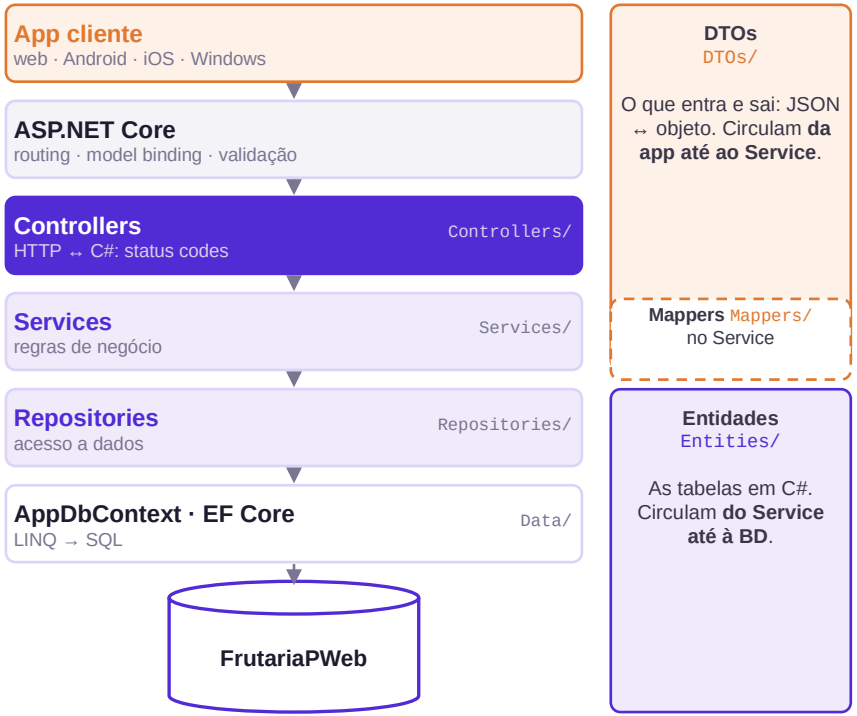



## A API não mexe na estrutura da BD

- **Nunca** `Add-Migration` nem `Update-Database` no projeto `RESTfulAPI`.
- As entidades são **cópias** das da Gestão de Loja: não se alteram aqui.
- Precisa de uma coluna nova? Muda-se na **Gestão de Loja** (entidade + migration) e só depois se copia a entidade para a API.
- A API só lê e grava **dados**, com o EF Core.

CAMADAS · ARQUITETURA

# A arquitetura da RESTfulAPI




## Regra 1: só para baixo

Cada camada chama apenas a de baixo. Um controller nunca usa o `AppDbContext`; um repository nunca escolhe um status code.

## Regra 2: por interfaces

O controller recebe um `IProdutoService`, e não um `ProdutoService`. Quem cria os objetos é o ASP.NET Core (injeção de dependências).

## Regra 3: a fronteira é o Service

Acima do Service só há DTOs; abaixo, só entidades. É no Service que o Mapper converte uma coisa na outra.

CAMADAS · RESPONSABILIDADES

# Quem faz o quê, e quem não pode fazer

| Camada | Recebe | Devolve | Conhece | Não conhece |
|---|---|---|---|---|
| **Controller** | pedido HTTP (já em DTO) | `ActionResult`: status + DTO | a interface do Service, os DTOs | `AppDbContext`, entidades, SQL |
| **DTO** | · | · | só os seus campos e validações | tudo o resto: é só dados |
| **Mapper** | entidade ou DTO | o outro | entidades e DTOs | base de dados, regras, HTTP |
| **Service** | DTO | `Resultado` com DTO | interfaces dos Repositories, Mappers | HTTP: status codes, `Request` |
| **Repository** | entidade, id, filtros | entidade(s) | `AppDbContext`, EF Core | DTOs, HTTP, regras de negócio |
| **AppDbContext** | LINQ | entidades | as tabelas e a connection string | quem o chama |

**Teste rápido:** se uma classe precisa de `using Microsoft.AspNetCore.Mvc` e de `using Microsoft.EntityFrameworkCore` ao mesmo tempo, está a fazer o trabalho de duas camadas.

ETAPAS 1 E 2 · O PROJETO

# A estrutura do projeto RESTfulAPI

Solution Explorer (as pastas a laranja são as nossas)

```
RESTfulAPI/
  Controllers/    ApiControllerBase.cs  CategoriasController.cs
                  ProdutosController.cs
                  ModosEntregaController.cs
  DTOs/           ProdutoDto.cs  CriarProdutoDto.cs
                  AtualizarProdutoDto.cs  ReferenciaDto.cs
                  ... (o mesmo para Categoria e Modo)
  Mappers/        ProdutoMapper.cs  CategoriaMapper.cs ...
  Services/       IProdutoService.cs  ProdutoService.cs
                  ...  Resultado.cs  Imagens.cs
  Repositories/   IProdutoRepository.cs  ProdutoRepository.cs ...
  Entities/       Categoria.cs  Produto.cs
                  ModoEntrega.cs  Morada.cs
  Data/           AppDbContext.cs  ApplicationUser.cs
                  VerificacaoBaseDados.cs
  Erros/          CodigosErro.cs  ErrosApi.cs
  appsettings.json   // ConnectionStrings:DefaultConnection
  Program.cs         // regista tudo e arranca
  RESTfulAPI.csproj  // pacotes NuGet e documentação XML
```

### O que a ficha pede (2.b e 2.c)

As pastas `Data`, `Entities` e `Repositories`, com as classes, as interfaces e os serviços.

### O que acrescentamos

`DTOs`, `Mappers`, `Services` e `Erros`: cada camada na sua pasta. As interfaces ficam ao lado da classe que as implementa.

### As entidades são cópias

As mesmas classes da Gestão de Loja (ficha 5, com o extra), com o namespace `RESTfulAPI.Entities`. Mudar uma obriga a mudar a outra.

CAMADAS · CONTROLLER

# Controller: o empregado de mesa

Controllers/ProdutosController.cs (excerto)

```
[ApiController]
[Route("api/produtos")]
public class ProdutosController(IProdutoService service)
    : ApiControllerBase
{
    // GET /api/produtos/7
    [HttpGet("{id:int}")]
    public async Task<ActionResult<ProdutoDto>> GetById(int id)
    {
        var r = await service.GetByIdAsync(id);
        if (!r.Sucesso) return Erro(r);   // 404, com o DTO de erro
        return Ok(r.Valor);               // 200 + JSON
    }

    // POST /api/produtos
    [HttpPost]
    public async Task<ActionResult<ProdutoDto>> Create(
        CriarProdutoDto dto)
    { ... }
}
```

### Faz

- Recebe o pedido, já convertido em DTO.
- Chama **um** método do Service.
- Escolhe o status code da resposta.

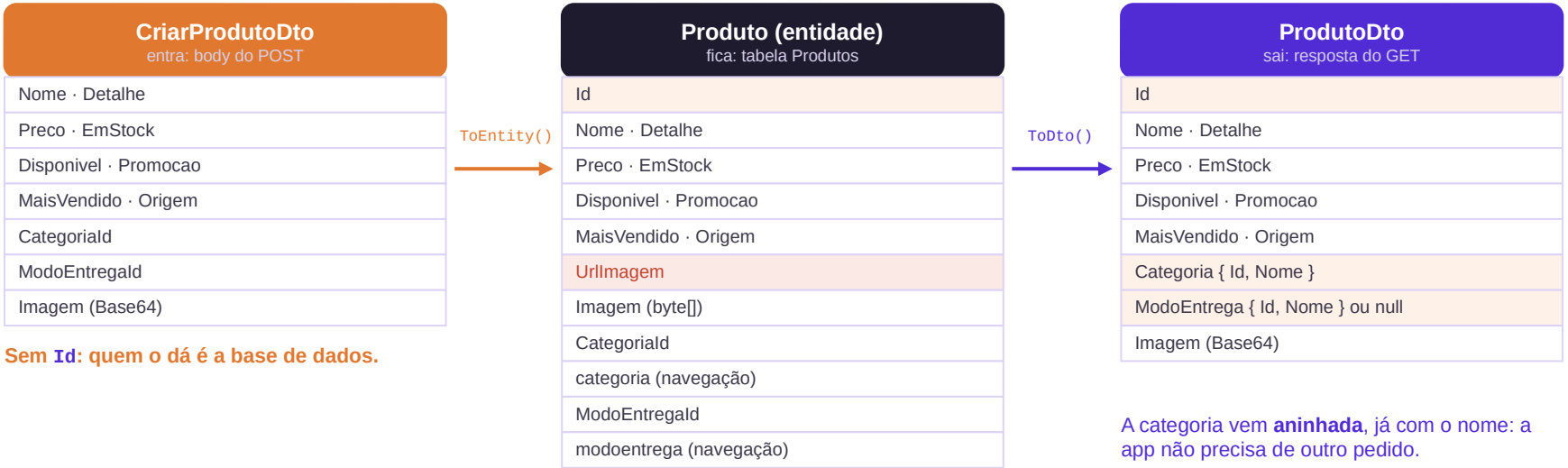
### Não faz

- SQL, `AppDbContext`, `Include`.
- Regras de negócio.
- Converter entidades em DTOs.

### `ControllerBase`, e não `Controller`

O `Controller` acrescenta as views (HTML) do MVC. Uma API não tem views.

# DTO: o que entra, o que fica, o que sai




**Sem `Id`: quem o dá é a base de dados.**

A categoria vem **aninhada**, já com o nome: a app não precisa de outro pedido.

campos da entidade que nunca saem

campos que só existem num dos lados

**DTO** = Data Transfer Object: uma classe só com propriedades, desenhada para **viajar**. Não tem métodos de negócio, nem navegações do EF Core.

DTO · SEGURANÇA

# Porquê DTOs: o ataque do campo a mais

O pedido de uma utilizadora espertinha

```
POST /api/auth/registar
Content-Type: application/json

{ "email": "eva@mail.pt", "password": "Ab12345!",
  "nome": "Eva",
  "estado": "Activo",
  "emailConfirmed": true }
```

**...com um DTO, só entra o que está previsto**

DTOs/RegistoDto.cs

```
public class RegistoDto
{
    public string Email { get; set; } = "";
    public string Password { get; set; } = "";
    public string Nome { get; set; } = "";
}
```

**Se o parâmetro for a entidade...**

```
public async Task<IActionResult> Registar(ApplicationUser u)
```

O model binding preenche **todas** as propriedades que vierem no JSON: `Estado = "Activo"`. A conta fica aprovada sem passar pelo Admin (o extra da ficha 5).

`estado` e `emailConfirmed` não existem no DTO: são **ignorados**. O Service cria o utilizador com `Estado = Pendente`, como manda a regra.

**O mesmo no POST de um produto**

Com a entidade, `"id": 999` no JSON faz o EF tentar inserir o Id 999: erro `500`.

DTO · PORQUÊ

# Porquê DTOs e não as entidades: quatro razões

### 1 · Segurança

Não sai o que é privado (`PasswordHash`, NIF). Não entra o que a app não pode mudar (`Estado`, `Id`).

### 2 · A forma certa

Só o que a app precisa, já pronto: a categoria com o nome, sem o UrlImagem nem as navegações do EF Core.

### 3 · Contrato estável

Renomear uma coluna na BD não parte a app Android que já está na loja. O DTO continua igual.

### 4 · JSON sem surpresas

Sem ciclos, sem `null` por falta de `Include`, sem `[JsonIgnore]` espalhados pelas entidades.

**A regra da nossa API:** nenhuma entidade passa do Service para cima. Os controllers recebem DTOs e devolvem DTOs. O preço: mais classes e um Mapper. O ganho: as quatro razões acima.

DTO · UM POR OPERAÇÃO

# Um DTO para cada operação

| Campo | ProdutoDto | CriarProdutoDto | AtualizarProdutoDto |
|---|---|---|---|
| `Id` | sim | · | no URL |
| `Nome`, `Detalhe`, `Preco`, `EmStock` | sim | sim | sim |
| `Disponivel`, `Promocao`, `MaisVendido` | sim | sim | sim |
| `Origem` | sim | sim | sim |
| `CategoriaId`, `ModoEntregaId` | · | sim | sim |
| `Categoria`, `ModoEntrega` (objetos) | sim | · | · |
| `Imagem` (Base64) | sim | sim | sim |
| validações (`[Required]`, `[Range]`) | · | sim | sim |
| **usado em** | GET (sai) | POST (entra) | PUT (entra) |

### Porque não um só?

Num só, o `Id` e a `Categoria` (objeto) estariam no POST, onde a app não os pode mandar.

### Criar e Atualizar iguais?

Hoje, quase. Separados, podem mudar cada um por si (um campo que só se define ao criar).

### O id do PUT

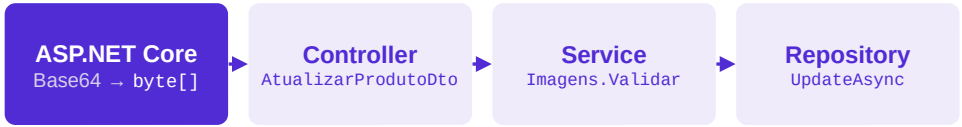
Vem só no URL. O DTO não tem `Id`: não há dois ids a discordar.

# Enviar uma imagem: no JSON, em Base64

O pedido HTTP (o que o Swagger, o Postman e a app enviam)

```
PUT /api/produtos/7 HTTP/1.1
Content-Type: application/json

{
  "nome": "Manga",
  "detalhe": "Manga madura, ao quilo",
  "preco": 3.49,
  ...
  "categoriaId": 1,
  "imagem": "iVBORw0KGgoAAAANSUhEUgAAAPAAAAB4CAYAAAA6..."
}
```




*O Base64 vira `byte[]` antes do controller (System.Text.Json). O Service verifica o tipo e os 2 MB.*

### Base64: a imagem em texto

O JSON só leva texto. Os bytes da imagem vão em Base64 (mais um terço), no campo `imagem`: o mesmo dos GET. Sem imagem: `null`.

### No Swagger e no Postman

O exemplo do Swagger já traz uma imagem pequena (16 x 16) em Base64. Para outra: converter num site (guia 3.d) e colar o Base64. Texto que não é Base64 dá `400 IMAGEM_INVALIDA`.

### O PUT substitui tudo

`"imagem": null` tira a imagem. Para a manter, a app volta a mandar a que recebeu no GET.

**Não é a forma mais correta** (slide 24): numa API a sério, a imagem iria à parte, como ficheiro.

CAMADAS · MAPPER

# Mapper: de entidade para DTO, e de volta

Mappers/ProdutoMapper.cs (excerto)

```
public static class ProdutoMapper
{
    // entidade → DTO de saída (GET)
    public static ProdutoDto ToDto(this Produto p) => new()
    {
        Id = p.Id, Nome = p.Nome!, Preco = p.Preco, ...
        Categoria = new ReferenciaDto
            { Id = p.categoria.Id, Nome = p.categoria.Nome! },
        Imagem = p.Imagem    // bytes → Base64 no JSON
    };

    // DTO de entrada → entidade nova (POST)
    public static Produto ToEntity(this CriarProdutoDto d) => new()
    {
        Nome = d.Nome, Preco = d.Preco, Imagem = d.Imagem, ...
        CategoriaId = d.CategoriaId, ModoEntregaId = d.ModoEntregaId
    };
}
```

### Só copia e converte

Sem regras, sem base de dados, sem HTTP. A imagem vai em bytes, tal como está.

### `this`: métodos de extensão

Escreve-se `produto.ToDto()`, como se o método fosse da classe `Produto`, sem mexer nela.

### Feito à mão

Há bibliotecas que o fazem sozinhas (AutoMapper, Mapster). À mão vê-se cada campo, e um campo esquecido salta à vista.

CAMADAS · VALIDAÇÃO

# Validação: três redes de segurança

### 1 · No DTO

**formato de cada campo**

`[Required]`, `[Range]`, `[StringLength]`, `IValidatableObject`

O `[ApiController]` responde **400** sozinho, **antes** de o controller correr.

### 2 · No Service

**regras que precisam da BD**

A categoria existe? O nome já está usado? A categoria tem produtos?

O Service devolve `Invalido` (**400**) ou `Conflito` (**409**).

### 3 · Na BD

**a última garantia**

`UNIQUE` no nome, `CHECK` no preço e no stock (extra da ficha 5).

Se tudo o resto falhar: `DbUpdateException`. Não se confia só nela.

DTOs/CriarProdutoDto.cs (excerto): a rede 1

```
[Range(0.01, 99999999.99, ErrorMessage = "O preço tem de ser maior do que zero.")]
public decimal Preco { get; set; }
```

Cada rede apanha o que a anterior **não consegue ver**: o DTO não sabe que nomes já existem.

CAMADAS · SERVICE

# Service: o chefe, que conhece as regras

Services/ProdutoService.cs (excerto)

```
public class ProdutoService(IProdutoRepository produtos,
    ICategoriaRepository categorias) : IProdutoService
{
    public async Task<Resultado<ProdutoDto>> CreateAsync(CriarProdutoDto dto)
    {
        // regras de negócio: precisam da base de dados
        if (!await categorias.ExisteAsync(dto.CategoriaId))
            return Resultado<ProdutoDto>.Invalido(CodigosErro
                .CategoriaInexistente, "A categoria não existe.");
        if (await produtos.NomeExisteAsync(dto.Nome))
            return Resultado<ProdutoDto>.Conflito(CodigosErro
                .ProdutoNomeRepetido, $"Já existe um produto \"{dto.Nome}\".");

        var produto = dto.ToEntity();          // Mapper
        await produtos.AddAsync(produto);       // INSERT
        var criado = await produtos.GetByIdAsync(produto.Id);
        return Resultado<ProdutoDto>.Ok(criado!.ToDto());
    }
}
```

## As regras da Frutaria

- A categoria e o modo têm de existir.
- Nomes únicos.
- Sem stock, não está disponível.
- A imagem: PNG, JPEG, GIF ou WebP, até 2 MB.
- Categoria com produtos não se apaga.

## `Resultado`, e não HTTP

O Service diz **o que** aconteceu (`NaoEncontrado`, `Invalido`, `Conflito`), com um **código** e uma **mensagem** para o utilizador. Não diz `404` nem `409`: isso é com o controller.

CAMADAS · REPOSITORY

# Repository: o único que entra na despensa

Repositories/IProdutoRepository.cs

```
public interface IProdutoRepository
{
    Task<List<Produto>> GetAllAsync(int? categoriaId,
        bool? promocao, string? pesquisa);
    Task<Produto?> GetByIdAsync(int id);
    Task<bool> NomeExisteAsync(string nome, int? excetoId = null);
    Task<bool> AddAsync(Produto produto);
    Task<bool> UpdateAsync(Produto produto);
    Task<bool> DeleteAsync(Produto produto);
}
```

Repositories/ProdutoRepository.cs (excerto)

```
public class ProdutoRepository(AppDbContext context)
    : IProdutoRepository
{
    public Task<Produto?> GetByIdAsync(int id) =>
        context.Produtos
            .Include(p => p.categoria)
            .Include(p => p.modoentrega)
            .FirstOrDefaultAsync(p => p.Id == id);
}
```

### Só aqui há EF Core

O `AppDbContext`, o `Include` e o `SaveChangesAsync` vivem nesta pasta e em mais nenhuma.

### `Include`: trazer a categoria

Gera um `JOIN`. Sem ele, `p.categoria` fica `null`.

### `AsNoTracking()` nas listagens

Só para ler: o EF não guarda cópias para comparar. Mais leve.

### Porquê a interface

O Service não sabe que é SQL Server. Num teste, troca-se o repository por um falso, em memória.

CAMADAS · INJEÇÃO DE DEPENDÊNCIAS

# Ninguém faz new: o ASP.NET Core monta tudo

Program.cs: o registo (uma vez, no arranque)

```
builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(connectionString));
builder.Services.AddScoped<IProdutoRepository, ProdutoRepository>();
builder.Services.AddScoped<ICategoriaRepository,
CategoriaRepository>();
builder.Services.AddScoped<IProdutoService, ProdutoService>();
```

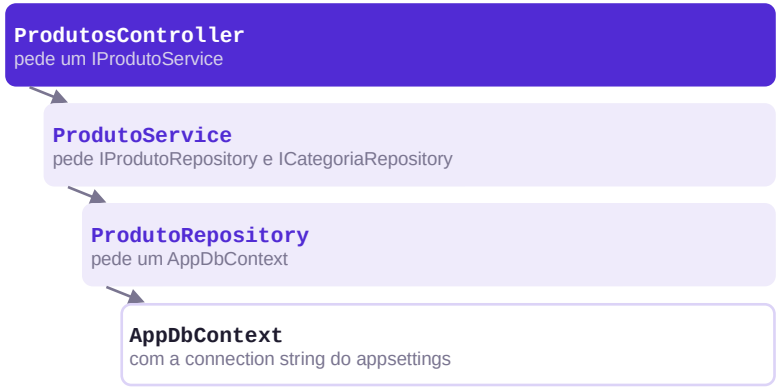
## AddScoped: um por pedido

Os objetos nascem com o pedido HTTP e morrem com a resposta. Um pedido, um `AppDbContext`.

## E a fábrica da ficha 5?

No Blazor a ligação dura minutos: precisámos da `IDbContextFactory`. Numa API, cada pedido é curto: o contexto injetado chega.

**Em cada pedido HTTP, o contentor cria:**




## Esquecer um registo

`Unable to resolve service for type 'IProdutoService'` no primeiro pedido: falta o `AddScoped`. Se faltar o de um Repository, a API nem arranca.

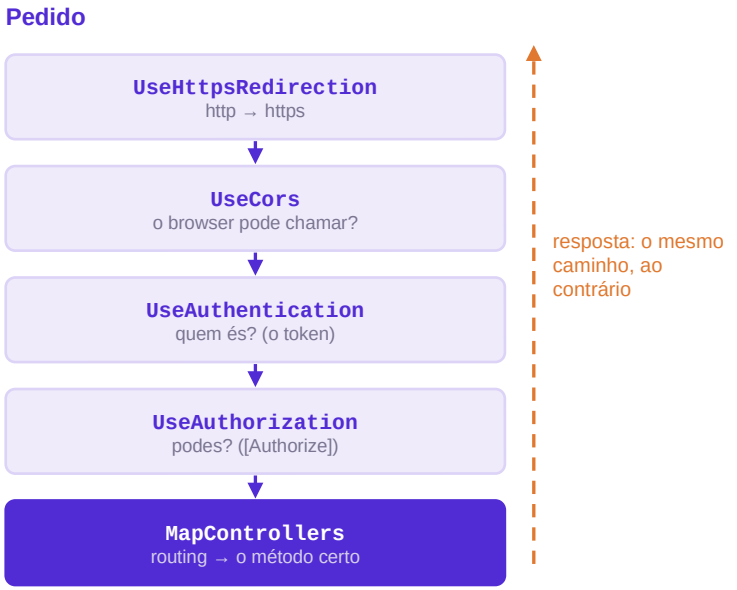
ETAPA 2.F · PROGRAM.CS

# Program.cs: os serviços e o caminho de cada pedido

Program.cs (resumo)

```
var builder = WebApplication.CreateBuilder(args);
// 1. serviços: o que o contentor sabe criar
builder.Services.AddDbContext<AppDbContext>(...);
builder.Services.AddScoped<...>();      // repos e services
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(...);   // + comentários ///

var app = builder.Build();
// 2. pipeline: por onde passa cada pedido, por ordem
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseCors();              // quando houver frontend web
app.UseAuthentication();    // próxima ficha
app.UseAuthorization();
app.MapControllers();
await VerificacaoBaseDados.ExecutarAsync(app); // sem BD, não arranca
app.Run();
```




**A ordem importa:** autenticar antes de autorizar.

# Do pedido ao método: rotas e model binding

O pedido

```
PUT /api/produtos/7?notificar=true
Content-Type: application/json

{ "nome": "Laranja", "preco": 1.59, ... }
```

O método que o recebe

```
[HttpPut("{id:int}")]
public async Task<IActionResult> Update(
    int id,                          // ← da rota
    [FromQuery] bool notificar,      // ← da query
    AtualizarProdutoDto dto)         // ← do body
```

O `[ApiController]` deduz a origem: tipos simples vêm da rota ou da query; classes vêm do body. Os atributos `[From...]` tornam-na explícita.

| Atributo | Onde se põe | O que faz |
|---|---|---|
| `[ApiController]` | na classe | 400 automático; deduz `[FromBody]`; obriga a rotas por atributo |
| `[Route("api/produtos")]` | na classe | o início do caminho de todos os métodos |
| `[HttpGet("{id:int}")]` | no método | verbo + resto do caminho; `:int` só aceita números (`/abc` dá 404) |
| `[FromRoute]` `[FromQuery]` `[FromBody]` | no parâmetro | de onde vem o valor: caminho, query string ou body |
| `[ProducesResponseType(404)]` | no método | documenta no Swagger os códigos possíveis |

CAMADAS · RESPOSTAS

# ActionResult: o controller escolhe o status code

| No controller | Status | Quando |
|---|---|---|
| `Ok(dto)` | 200 | leitura com sucesso |
| `CreatedAtAction(...)` | 201 | POST: com `Location` |
| `NoContent()` | 204 | PUT, DELETE |
| `ValidationProblem()` | 400 | automático |
| `NotFound()` | 404 | não existe |
| `Problem(msg, statusCode: 409)` | 409 | conflito |
| `Unauthorized()` · `Forbid()` | 401 · 403 | o `[Authorize]` faz |

### `ActionResult<ProdutoDto>`

O `<ProdutoDto>` diz ao Swagger que forma tem o JSON de sucesso. Sem ele, o Swagger não mostra o exemplo.

```
POST: sucesso ou erro, em três linhas
[HttpPost]
public async Task<ActionResult<ProdutoDto>>
    Create(CriarProdutoDto dto)
{
    var r = await service.CreateAsync(dto);
    if (!r.Sucesso) return Erro(r);  // 400/404/409
    return CreatedAtAction(nameof(GetById),
        new { id = r.Valor!.Id }, r.Valor);
}
// Erro(r) está na ApiControllerBase:
// NaoEncontrado→404, Invalido→400, Conflito→409
```

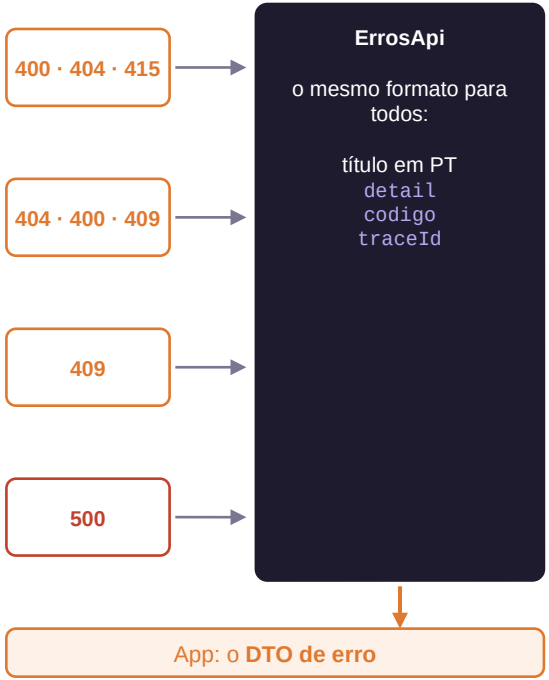
### `CreatedAtAction`

Constrói o header `Location` a partir do método `GetById`: `/api/produtos/20`.

ERROS · O PERCURSO

# Erros: de onde vêm, e como chegam à app

| Onde nasce | Exemplo | Como sobe |
|---|---|---|
| **ASP.NET Core**<br>antes do controller | preço `0`; `"preco": "abc"`; JSON mal escrito; rota que não existe | responde sozinho |
| **Service**<br>regras de negócio | o produto não existe; nome repetido; categoria com produtos | devolve `Resultado` com **código** e **mensagem**; o controller faz `Erro(r)` |
| **Repository**<br>a base de dados | a BD recusa a gravação (índice único, CHECK) | apanha a `DbUpdateException`, devolve `false`; o Service devolve `Conflito` |
| **Qualquer camada**<br>sem querer | connection string errada; um `null` inesperado | a **exceção** sobe sozinha até ao `UseExceptionHandler` |




*Cada camada fala na sua língua: só o controller (e o ASP.NET Core) falam HTTP.*

ERROS · DUAS FAMÍLIAS

# Esperado ou inesperado: Resultado ou exceção

### Esperado: faz parte do negócio

Um produto que não existe, um nome repetido. Acontece todos os dias. **Não** é uma exceção: o Service devolve um `Resultado`.

### Inesperado: algo partiu

A base de dados em baixo, um bug. Lança-se (ou rebenta) uma **exceção**, e ninguém a apanha pelo caminho.

Service → Controller

```
// ProdutoService
if (produto is null)
    return Resultado<ProdutoDto>.NaoEncontrado(
        CodigosErro.ProdutoNaoEncontrado,
        $"O produto {id} não existe.");
// ProdutosController
if (!r.Sucesso) return Erro(r);   // → 404
```

Program.cs: o handler global

```
builder.Services.AddProblemDetails(o =>
    o.CustomizeProblemDetails = ErrosApi.Personalizar);
...
app.UseExceptionHandler();   // exceção → 500
app.UseStatusCodePages();    // 404 de rota → corpo
```

### Porquê não exceções para tudo

Uma exceção é cara de ler: salta camadas sem se ver no código. O `Resultado` está na assinatura do método: quem chama sabe que pode falhar.

### Outra forma comum

Exceções próprias (`NaoEncontradoException`) + um `IExceptionHandler` que as traduz. Também é válida; nunca as duas misturadas.

ERROS · O CONTRATO

# O DTO de erro: o que a app recebe, sempre

400: validação (automático)

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "Os dados enviados têm erros.",
  "status": 400,
  "detail": "Corrija os campos assinalados e tente outra vez.",
  "errors": {
    "preco": [ "O preço tem de ser maior do que zero." ] },
  "codigo": "DADOS_INVALIDOS",
  "traceId": "0HN7Q3R2K9S1A:00000005"
}
```

409: regra de negócio (do Service)

```
{
  "title": "Conflito com os dados existentes.",
  "status": 409,
  "detail": "Já existe um produto \"Manga\".",
  "codigo": "PRODUTO_NOME_REPETIDO",
  "traceId": "0HN7Q3R2K9S1A:00000004"
}
```

| Campo | É... | A app usa-o para... |
|---|---|---|
| `status` | o status code | decidir o tipo de reação |
| `title` | o tipo de erro | um título curto |
| `detail` | **a frase em PT** | mostrar ao utilizador |
| `codigo` | estável, nunca muda | o **código** da app: `if (codigo == ...)` |
| `errors` | só nos 400 | pôr cada erro junto ao seu campo |
| `traceId` | o id do pedido | o suporte encontrar o erro no log |

### Porquê ProblemDetails

É o padrão (RFC 9457) e o ASP.NET Core usa-o nos erros que gera sozinho. Personalizado num sítio, **todos** os erros ficam iguais.

ERROS · EM CÓDIGO

# Um só sítio para o formato: o ErrosApi

Erros/ErrosApi.cs (resumo)

```
public static void Personalizar(ProblemDetailsContext contexto)
{
    var pd = contexto.ProblemDetails;
    var (titulo, codigo, detalhe) = pd.Status switch
    {
        400 => ("Os dados enviados têm erros.", "DADOS_INVALIDOS", ...),
        404 => ("Não encontrado.", "NAO_ENCONTRADO", ...),
        409 => ("Conflito com os dados existentes.", "CONFLITO", ...),
        _   => ("Erro interno do servidor.", "ERRO_INTERNO", ...)
    };
    pd.Title = titulo;
    if (pd.Status >= 500 || pd.Detail is null) pd.Detail = detalhe;
    // o código específico do Service, se o controller o deixou
    if (contexto.HttpContext.Items["codigoErro"] is string c) codigo = c;
    pd.Extensions["codigo"] = codigo;
    pd.Extensions["traceId"] = contexto.HttpContext.TraceIdentifier;
}
```

No controller, em Erro(r):

```
HttpContext.Items["codigoErro"] = r.Codigo;  return Problem(...);
```

### Quem o chama

O ASP.NET Core, **sempre** que cria um ProblemDetails: nos nossos `Problem()`, nos 400 automáticos, nos 404 de rota, nos 500.

### Nos 500, nada de detalhes

O `detail` é sempre "Ocorreu um erro inesperado". O stack trace fica **só** na consola da API.

### Os códigos

Ficam em `Erros/CodigosErro.cs`, como constantes: `CodigosErro.ProdutoNomeRepetido`. Um engano no nome dá erro de compilação, e não um código errado que a app não reconhece.

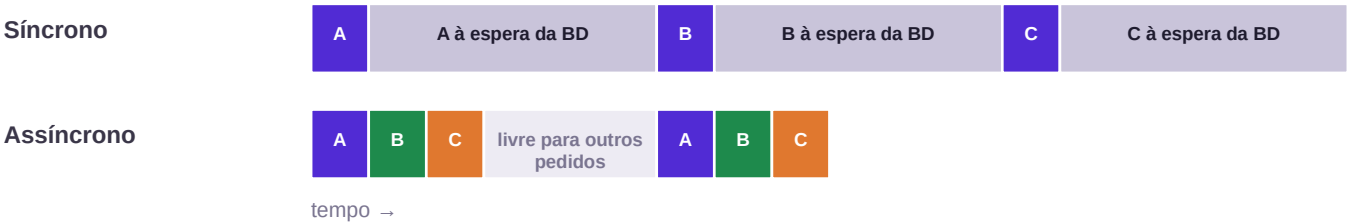
ERROS · OS CÓDIGOS

# Os códigos de erro da RESTfulAPI

| Status | codigo | Quando |
|---|---|---|
| 400 | `DADOS_INVALIDOS` | falhou a validação do DTO, ou um campo tem o tipo errado (os detalhes vêm em `errors`) |
| 400 | `JSON_INVALIDO` | o corpo do pedido não é JSON válido |
| 400 | `CATEGORIA_INEXISTENTE` · `MODO_ENTREGA_INEXISTENTE` | o produto aponta para uma categoria ou um modo que não existe |
| 400 | `IMAGEM_INVALIDA` · `IMAGEM_DEMASIADO_GRANDE` | no POST e no PUT: o campo `imagem` não é Base64, não é uma imagem, ou tem mais de 2 MB |
| 404 | `PRODUTO_NAO_ENCONTRADO` · `CATEGORIA_...` · `MODO_ENTREGA_...` | o recurso pedido no URL não existe |
| 404 | `NAO_ENCONTRADO` | rota que não existe (por exemplo /api/naoexiste) |
| 409 | `PRODUTO_NOME_REPETIDO` · `CATEGORIA_...` · `MODO_ENTREGA_...` | já existe outro com o mesmo nome |
| 409 | `CATEGORIA_COM_PRODUTOS` · `MODO_ENTREGA_EM_USO` | não se apaga: tem produtos associados |
| 409 | `GRAVACAO_RECUSADA` | a base de dados recusou (por exemplo, dois pedidos ao mesmo tempo) |
| 415 | `FORMATO_NAO_SUPORTADO` | faltou `Content-Type: application/json` |
| 500 | `ERRO_INTERNO` | exceção inesperada: ver a consola da API |
| 401 · 403 | `NAO_AUTENTICADO` · `SEM_PERMISSAO` | próxima ficha, com o JWT |

# async e await: o empregado não fica parado à espera

**Uma thread (um empregado) e três pedidos; a base de dados demora 50 ms a responder**




O padrão, em todas as camadas

```
public async Task<ActionResult<ProdutoDto>> GetById(int id)
{
    // await: "vai à BD; quando houver resposta,
    // continuo daqui". A thread fica livre.
    var r = await service.GetByIdAsync(id);
    ...
}
```

### Async até ao fundo

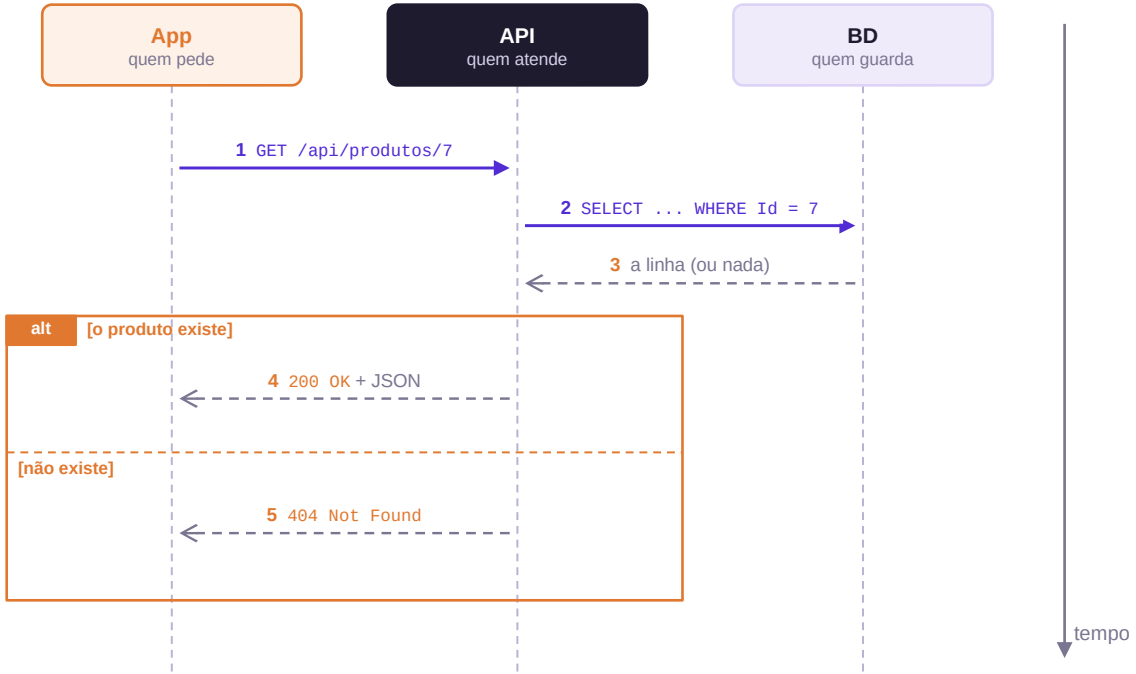
Controller, Service, Repository: todos `async Task<...>`, todos com `await`.

### Nunca `.Result` nem `.Wait()`

Bloqueiam a thread à espera: perde-se a vantagem, e pode encravar.

DIAGRAMAS DE SEQUÊNCIA

# Como ler um diagrama de sequência




**Participantes**
As caixas de cima: as camadas (ou programas). A linha tracejada é a **linha de vida**.

**Mensagem**
Seta cheia: uma chamada (um pedido, um método). O número dá a ordem.

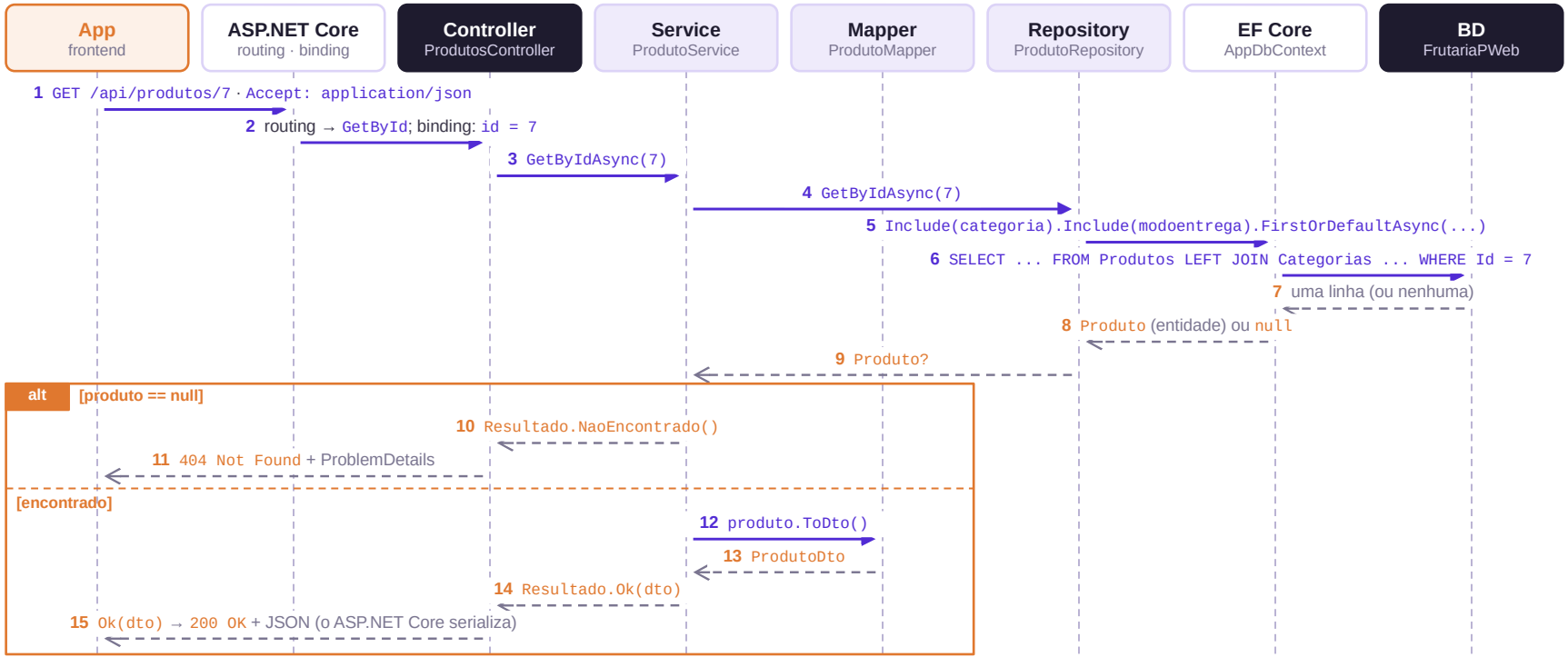
**Resposta**
Seta tracejada: o que volta. Número a laranja.

**alt**
Caminhos alternativos: só acontece **um** dos blocos, conforme a condição `[...]`.

**ref**
Um pedaço já desenhado noutro diagrama, resumido numa caixa.

DO PEDIDO À BD · GET

# GET /api/produtos/7: ler um produto



DO PEDIDO À BD · GET

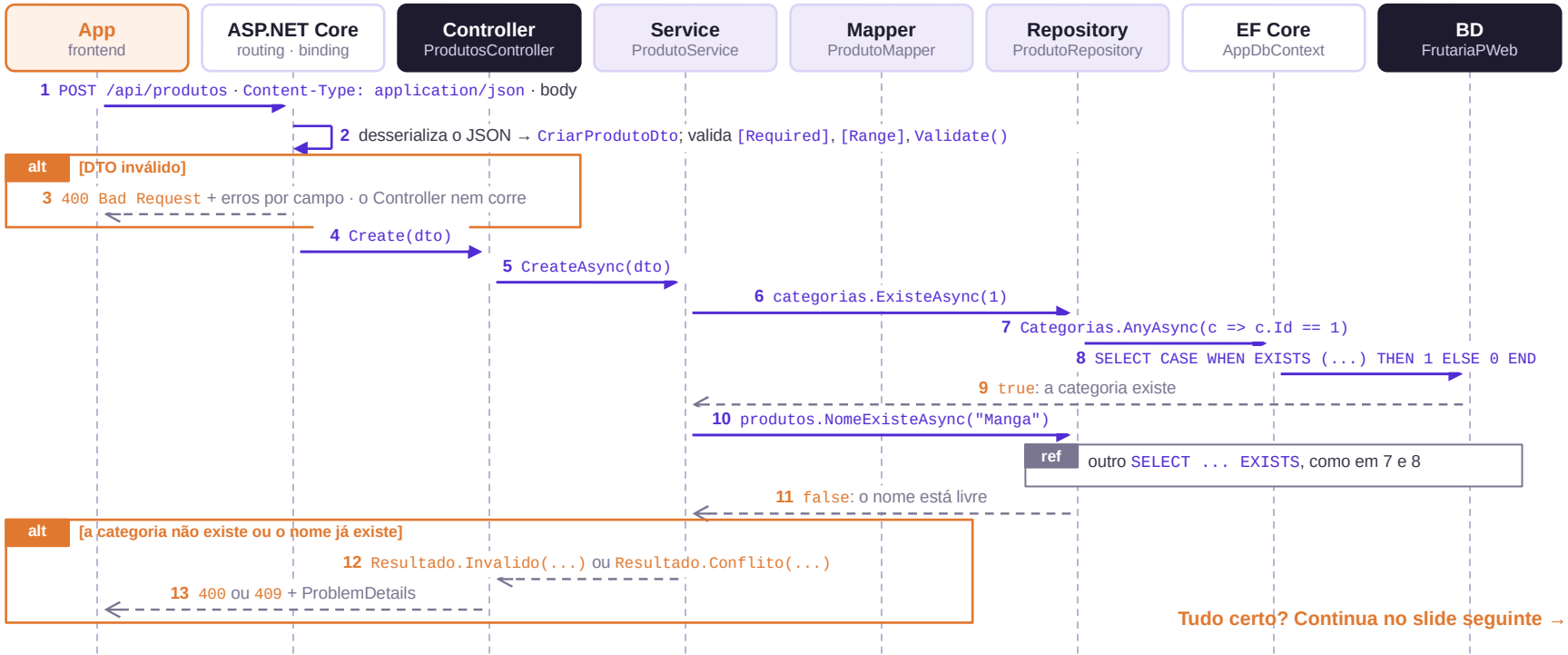
# GET passo a passo: o que viaja em cada seta

| # | Onde | O que acontece | O que viaja |
|---|---|---|---|
| 1 | App → ASP.NET Core | A app pede o produto 7 | texto HTTP: `GET /api/produtos/7` |
| 2 | ASP.NET Core | Routing escolhe o método; o "7" (texto) passa a `int` | `int id = 7` |
| 3-4 | Controller → Service → Repository | Cada camada passa o pedido à de baixo | `7` |
| 5-6 | EF Core → BD | O LINQ é traduzido em SQL, com `JOIN` por causa do `Include` | `SELECT ... WHERE [p].[Id] = @id` |
| 7-9 | BD → Repository → Service | A linha da tabela vira um objeto | entidade `Produto` (com os bytes da imagem) |
| 10-11 | Service → Controller → App | Não existe: o Service diz-o, o Controller traduz | `NaoEncontrado` → `404` |
| 12-13 | Service ↔ Mapper | A entidade é convertida em DTO; a imagem vai em bytes (Base64 no JSON) | `ProdutoDto` |
| 14-15 | Service → Controller → App | O DTO é serializado em JSON e enviado | `200 OK` + `{ "id": 7, "nome": "Uvas", ... }` |

**GET /api/produtos** (a lista) faz o mesmo caminho: o Repository devolve uma `List<Produto>`, com `AsNoTracking()`, e não há 404, há uma lista vazia `[]`.

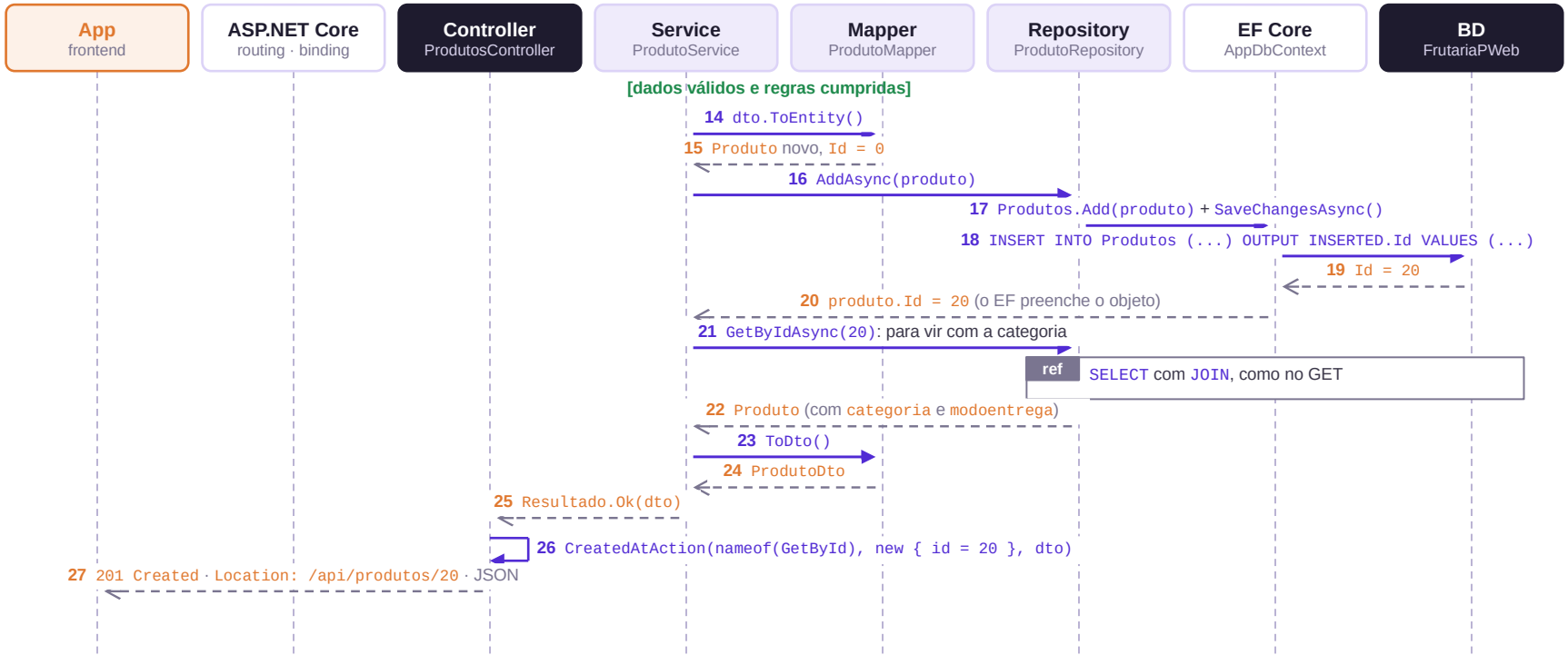
DO PEDIDO À BD · POST (1/2)

# POST /api/produtos: primeiro, validar




Tudo certo? Continua no slide seguinte →

# POST /api/produtos: depois, gravar



DO PEDIDO À BD · POST

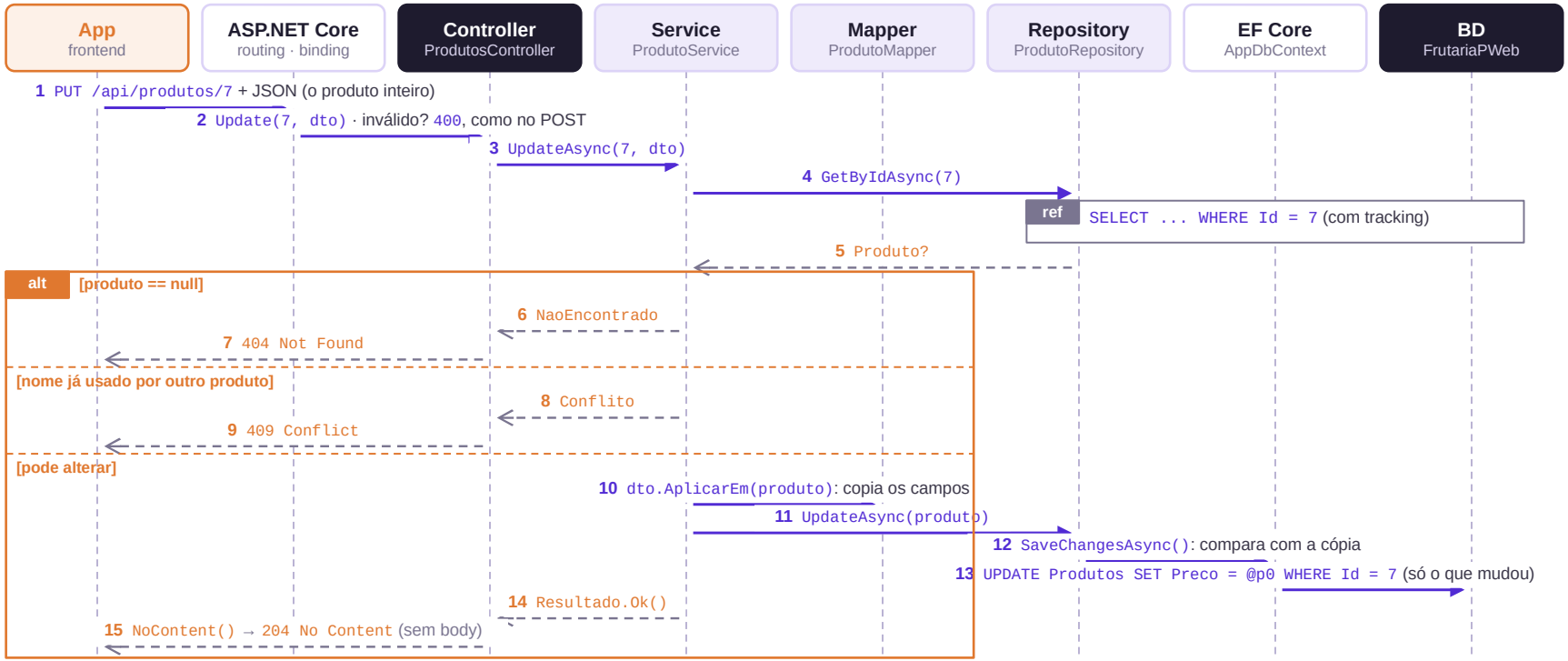
# POST passo a passo: o que viaja em cada seta

| # | Onde | O que acontece | O que viaja |
|---|---|---|---|
| 1 | App → ASP.NET Core | A app manda o produto novo, sem id | texto HTTP com JSON no body |
| 2-3 | ASP.NET Core | JSON → DTO; validação de formato; se falhar, `400` | `CriarProdutoDto` ou `ValidationProblemDetails` |
| 4-11 | Controller → Service ↔ Repository | Regras com dados: a categoria existe? o nome está livre? | `1`, `"Manga"` → `true` / `false` |
| 12-13 | Service → Controller → App | Uma regra falhou | `Invalido` → `400`; `Conflito` → `409` |
| 14-15 | Service ↔ Mapper | DTO → entidade nova | `Produto { Id = 0, CategoriaId = 1, ... }` |
| 16-20 | Repository → EF Core → BD | Add + SaveChangesAsync: o INSERT, e o Id de volta | `INSERT ...` → `Id = 20` |
| 21-24 | Service, Repository, Mapper | Reler com `Include` e converter em DTO | `ProdutoDto` com a categoria |
| 25-27 | Service → Controller → App | Sucesso: `201` com o endereço do novo | `Location: /api/produtos/20` + JSON |

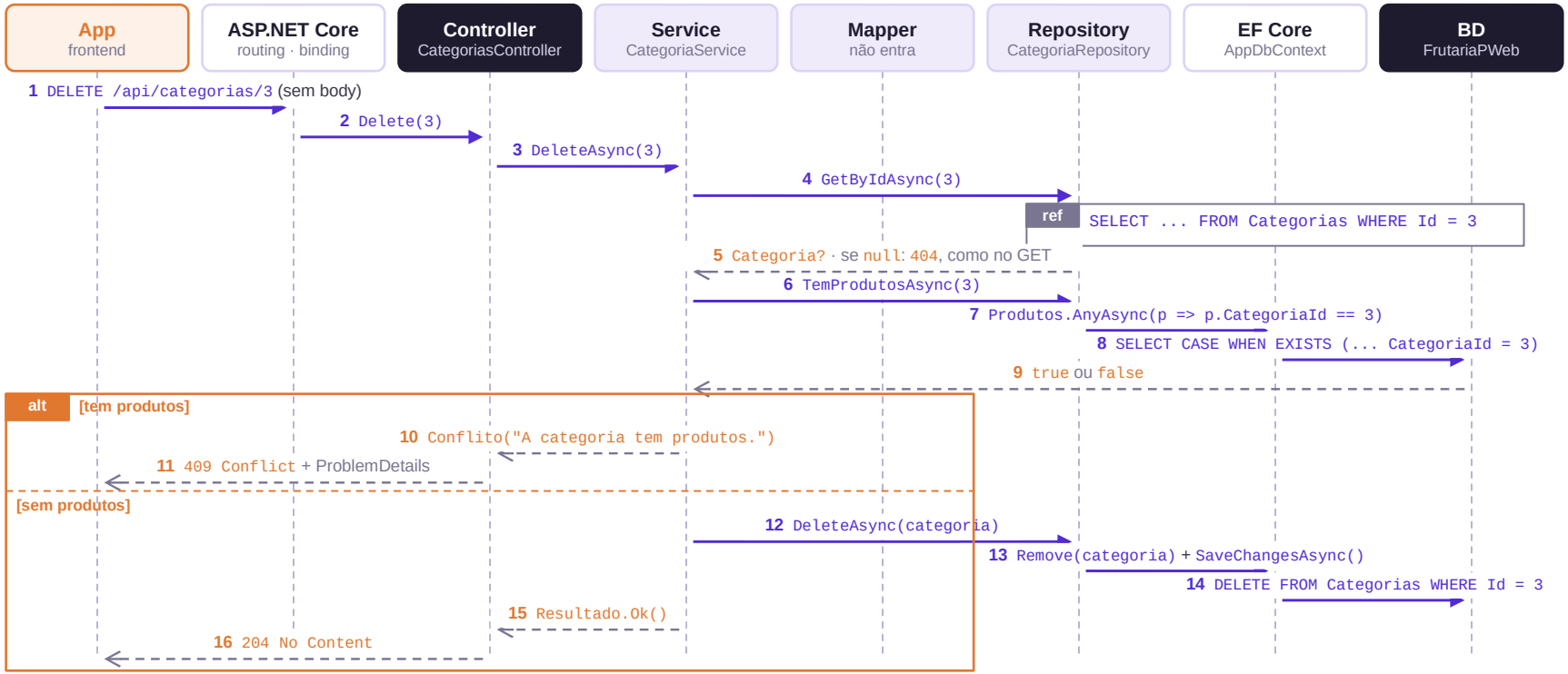
**E se dois pedidos chegarem ao mesmo tempo com o mesmo nome?** Os dois passam a regra 10; o índice único da BD trava o segundo (`DbUpdateException`).

DO PEDIDO À BD · PUT

# PUT /api/produtos/7: alterar um produto

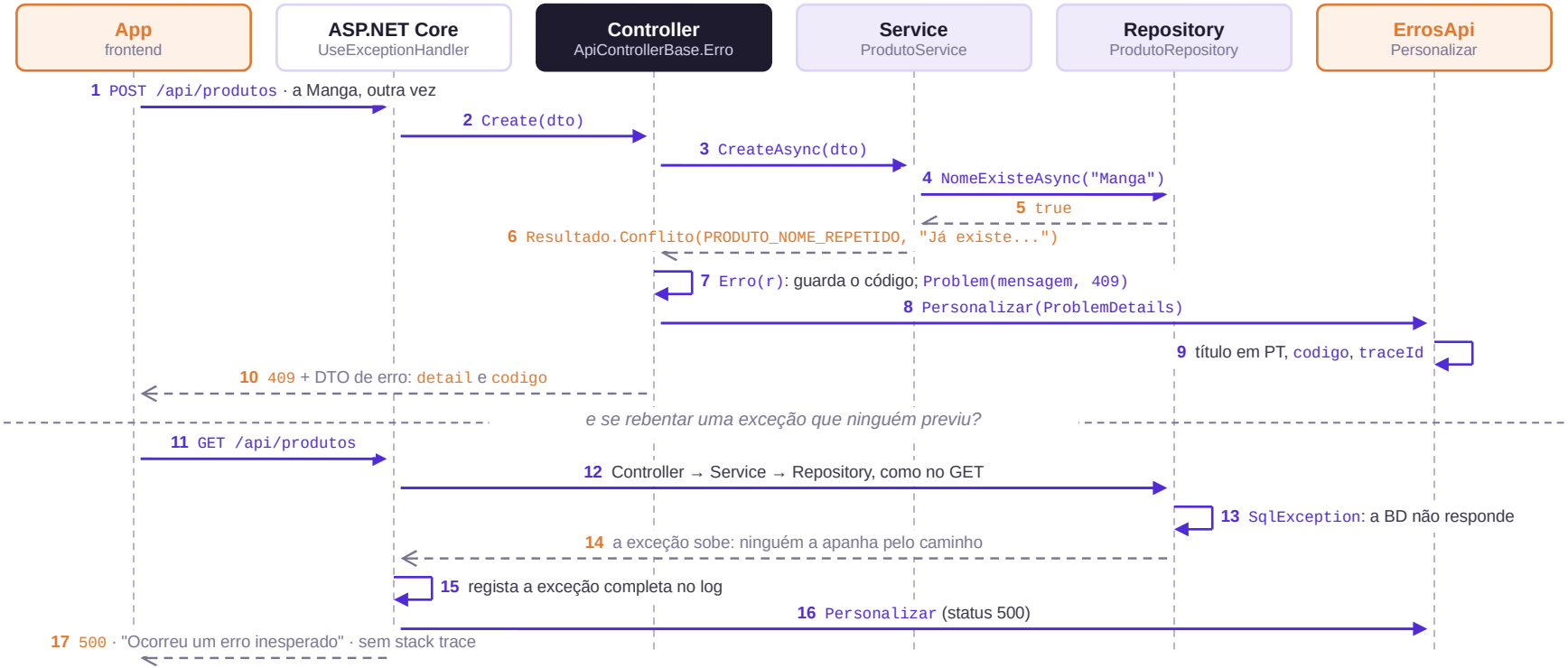


# DELETE /api/categorias/3: apagar, se puder



DO PEDIDO À BD · ERROS

# Um erro do princípio ao fim: 409 e 500



DO PEDIDO À BD · RESUMO

# Cada camada, em cada verbo

| | GET (ler) | POST (criar) | PUT (alterar) | DELETE (apagar) |
|---|---|---|---|---|
| **ASP.NET Core** | id da rota | JSON → DTO; valida | id + JSON → DTO; valida | id da rota |
| **Controller** | `Ok` ou `NotFound` | `CreatedAtAction` | `NoContent` | `NoContent` |
| **Service** | existe? | categoria? nome livre? | existe? nome livre? | existe? tem produtos? |
| **Mapper** | `ToDto` | `ToEntity`, `ToDto` | `AplicarEm` | · |
| **Repository** | `GetByIdAsync` | `AddAsync` | `UpdateAsync` | `DeleteAsync` |
| **SQL** | `SELECT` | `INSERT` | `UPDATE` | `DELETE` |
| **Sucesso** | `200` + JSON | `201` + Location + JSON | `204` | `204` |
| **Erros** | `404` | `400`, `409` | `400`, `404`, `409` | `404`, `409` |

Lê-se em coluna (um verbo, de cima para baixo) ou em linha (uma camada, nos quatro verbos).

PARTE PRÁTICA · ETAPAS 1 A 3 DA FICHA

# Mãos à obra: a RESTfulAPI, alínea a alínea

Cada slide verde é uma etapa da aula: o que fazer, os ficheiros a criar ou analisar, o que perceber, e como saber que correu bem. O código não está aqui: está no guia HTML, na secção indicada no canto de cada slide.

**Legenda dos ficheiros**

| | |
|---|---|
| + | criar |
| ✎ | alterar |
| × | apagar |
| = | fica igual |
| » | abrir e analisar |

PRÁTICA · O MAPA

# O caminho da aula prática

**Etapa 1** — criar

- 1.a — Criar o projeto
- 1.b — ✓ **Compilar e ver o Swagger**

**Etapa 2** — organizar

- 2.a — Limpar
- 2.b — Pastas e NuGet
- 2.c — Entities a Services, de baixo para cima
- 2.d — ✓ **Compilar**
- 2.e–2.f — appsettings e Program.cs
- 2.g — ✓ **Compilar**
- 2.h — Controllers
- 2.i — ✓ **Compilar**
- 2.j — ✓ **Swagger**

**Etapa 3** — expor e testar

- 3.a–3.b — Dev Tunnel
- 3.c — ✓ **Testar: Swagger e Postman**

**Extra** — fora do enunciado

- E — Logging

**As caixas verdes são pontos de controlo:** compila, ou o Swagger mostra o que deve. Ninguém avança com erros por resolver.

PRÁTICA · A ORDEM DA ETAPA 2.C

# Construir de baixo para cima

| | | | |
|---|---|---|---|
| 8 | **Controllers** | 2.h | chamam os Services |
| 7 | **Program.cs** | 2.f | regista tudo o que está abaixo |
| 6 | **Services + Erros** | 2.c.6 | usam Repositories e Mappers |
| 5 | **Repositories** | 2.c.5 | usam o AppDbContext |
| 4 | **Mappers** | 2.c.4 | usam entidades e DTOs |
| 3 | **DTOs** | 2.c.3 | não usam ninguém |
| 2 | **Data** | 2.c.2 | usa as entidades |
| 1 | **Entities** | 2.c.1 | não usam ninguém |

a ordem da aula

## Porquê esta ordem

Cada ficheiro só usa classes que **já existem**. Assim o projeto compila a cada passo, e um erro aponta sempre para o ficheiro acabado de criar.

## É o contrário do pedido

Um pedido entra pelo Controller e desce até à base de dados (slide 28). Construímos ao contrário: primeiro os alicerces.

## Truque

Ctrl+Shift+B depois de cada pasta, mesmo onde a ficha não pede. Um erro de cada vez.

# Criar o projeto e ver o primeiro Swagger

## Fazer

- Visual Studio > Criar um projeto > **API Web do ASP.NET Core** (selo C#).
- Nome `RESTfulAPI`, e as opções do slide seguinte.
- Compilar (Ctrl+Shift+B) e executar (F5, botão ► https).
- No Swagger: `GET /WeatherForecast` > Try it out > Execute.

## Ficheiros

**gerados pelo template**

» `Program.cs` · onde arranca
» `Properties/launchSettings.json` · portas
» `Controllers/WeatherForecastController.cs`
= `WeatherForecast.cs`
= `RESTfulAPI.http`
= `appsettings.json`

## Explicar

- **O template é um esqueleto** que já funciona: um controller, um endpoint, o Swagger.
- A janela preta é o servidor (Kestrel): `Now listening on: https://localhost:7xxx`.
- **Swagger** = a ementa da API, gerada a partir do código, com um botão para experimentar.
- No `WeatherForecastController`: `[HttpGet]` + um método que devolve objetos C#, e o JSON aparece sozinho (slide 23).

✔ **Ponto de controlo.** o Swagger abre em `/swagger` e `GET /WeatherForecast` responde `200` com cinco previsões em JSON.

PRÁTICA · ALÍNEA 1.A

# Criar o projeto: as opções que contam

**Informações adicionais** · API Web do ASP.NET Core

| | |
|---|---|
| Estrutura | **.NET 8.0 (Suporte de Longo Prazo)** |
| Tipo de autenticação | **Nenhum** |

- [x] Configurar para HTTPS
- [ ] Habilitar o suporte a contêineres
- [x] Habilitar o suporte a OpenAPI
- [ ] Não use instruções de nível superior
- [x] **Usar controles**
- [ ] Inscreva-se na orquestração do .NET Aspire

### Template e nome

**API Web do ASP.NET Core**, o do selo C# (não o "native AOT"). Nome: `RESTfulAPI`. As figuras da ficha mostram `RESTfulAPIPWeb`; vale o texto.

### Usar controles: obrigatório

Sem esta opção, o template usa Minimal APIs (`app.MapGet(...)` no `Program.cs`), sem a pasta `Controllers`.

### OpenAPI: o Swagger vem incluído

No .NET 8, o template instala o Swashbuckle e liga o Swagger.

### Autenticação: Nenhum

O login com JWT entra à mão, na próxima ficha.

# Limpar o projeto: sai o exemplo do tempo

## Fazer

- No Gerenciador de Soluções, clique direito > **Excluir** em cada um dos três ficheiros.
- A pasta `Controllers` **fica**, vazia.
- Compilar.

## Ficheiros

- ✕ `WeatherForecast.cs`
- ✕ `Controllers/WeatherForecastController.cs`
- ✕ `RESTfulAPI.http`

**ficam, sem mexer**

= `Controllers/` · vazia até à 2.h
= `Program.cs` · muda na 2.f
= `appsettings.json` · muda na 2.e
= `Properties/launchSettings.json`

## Explicar

- Eram só um exemplo: dados inventados (previsões do tempo ao acaso).
- O `Program.cs` do template não fala do `WeatherForecast`: apagar os três não parte nada.
- O `.http` testava pedidos dentro do Visual Studio; nós usamos o Swagger e o Postman.

✓ **Ponto de controlo.** compila com 0 erros. Se executarem, o Swagger diz "No operations defined in spec!": é normal, não há endpoints.

# Uma pasta por camada, e dois pacotes NuGet

`Controllers/` **DTOs/** **Mappers/** **Services/** **Erros/** **Repositories/** **Data/** **Entities/**

## Fazer

- Clique direito no projeto > Adicionar > Nova Pasta, sete vezes.
- Ferramentas > Gerenciador de Pacotes do NuGet > **Console**.
- Os dois `Install-Package` do guia, um de cada vez, versão **8.0.10**. Depois, no **.csproj**, as duas linhas da documentação XML.

## Ficheiros

**pastas da ficha**

+ `Data/`
+ `Entities/`
+ `Repositories/`

**pastas do guia**

+ `DTOs/`
+ `Mappers/`
+ `Services/`
+ `Erros/`

**alterado pelo NuGet**

✎ `RESTfulAPI.csproj` · 3 pacotes + documentação XML

## Explicar

- **Pasta = camada = namespace**: `Services/` dá `namespace RESTfulAPI.Services`.
- `EntityFrameworkCore.SqlServer`: o EF Core e o `UseSqlServer`.
- `Identity.EntityFrameworkCore`: o `IdentityDbContext`, porque a base de dados já tem as tabelas `AspNetUsers`...
- **8.0.10 porque o projeto é .NET 8.** Um pacote 9.x não instala.
- As 4 pastas a mais: slide 30.

✔ **Ponto de controlo.** duplo clique no projeto abre o `.csproj`, com três `PackageReference` (Swashbuckle, SqlServer, Identity).

# Entities: as mesmas classes da Gestão de Loja

Controllers/ | DTOs/ | Mappers/ | Services/ | Erros/ | Repositories/ | Data/ | **Entities/**

## Fazer

- Na pasta `Entities`: Adicionar > Classe, para cada ficheiro.
- Substituir o conteúdo pelo do guia.
- Abrir ao lado o `Produto.cs` da **Gestão de Loja** (ficha 5) e comparar.

## Ficheiros

+ `Entities/Categoria.cs`
+ `Entities/ModoEntrega.cs`
+ `Entities/Morada.cs`
+ `Entities/Produto.cs`

**comparar com**

» `GestaoLoja/Entities/Produto.cs`

## Explicar

- **Mesma base de dados, mesmas classes**: o EF Core lê as propriedades para saber as colunas. Uma a mais dá `Invalid column name`.
- Muda o namespace (`RESTfulAPI.Entities`) e sai o `[JsonIgnore]`: na API, as entidades nunca viram JSON.
- `Produto` valida-se a si mesmo (`IValidatableObject`): sem stock não pode estar disponível.
- Ligação à ficha 5: slide 16.

✓ **Ponto de controlo.** Ctrl+Shift+B compila (opcional, mas ajuda a apanhar gralhas cedo).

# Data: o contexto, a ponte para a base de dados

Controllers/ | DTOs/ | Mappers/ | Services/ | Erros/ | Repositories/ | **Data/** | Entities/

## Fazer

- Na pasta `Data`: `ApplicationUser.cs`, `AppDbContext.cs` e `VerificacaoBaseDados.cs`.
- Ler o `OnModelCreating` com a turma.
- **Não** fazer `Add-Migration`.

## Ficheiros

+ `Data/ApplicationUser.cs`
+ `Data/AppDbContext.cs`
+ `Data/VerificacaoBaseDados.cs`

**comparar com**

» `GestaoLoja/Data/ApplicationDbContext.cs`

## Explicar

- `AppDbContext` herda de `IdentityDbContext<ApplicationUser>`: a base de dados tem as tabelas do Identity, e o contexto tem de as conhecer.
- Cada `DbSet` é uma tabela: `Produtos`, `Categorias`, `ModosEntrega`, `Moradas`.
- **Sem migrations**: o esquema é da Gestão de Loja (slide 3).
- `VerificacaoBaseDados`: sem base de dados, ou sem as tabelas certas, a API **não arranca**, e a consola diz porquê.

✓ **Ponto de controlo.** compila. Ainda não há ligação à base de dados: a connection string só entra na 2.e, e a verificação só corre a partir da 2.f.

# DTOs: o JSON que entra e o que sai

Controllers/ | **DTOs/** | Mappers/ | Services/ | Erros/ | Repositories/ | Data/ | Entities/

## Fazer

- Na pasta `DTOs`, dez classes. Começar pela pequena: `ReferenciaDto`.
- Depois três por recurso: `...Dto`, `Criar...Dto`, `Atualizar...Dto`.
- Abrir lado a lado `Produto.cs`, `ProdutoDto.cs` e `CriarProdutoDto.cs`.

## Ficheiros

+ `DTOs/ReferenciaDto.cs`
+ `DTOs/CategoriaDto.cs` · e Criar..., Atualizar...
+ `DTOs/ModoEntregaDto.cs` · e Criar..., Atualizar...
+ `DTOs/ProdutoDto.cs` · e Criar..., Atualizar...

**analisar lado a lado**

» `Entities/Produto.cs`
» `DTOs/ProdutoDto.cs`
» `DTOs/CriarProdutoDto.cs`

## Explicar

- **Um DTO por operação**: o de saída tem `Id`; os de entrada não (slide 35).
- A imagem vai em bytes (`imagem`, Base64), a entrar e a sair; `null` = sem imagem. **Não é a forma mais correta**: torna as listagens pesadas.
- Em vez de `CategoriaId`, leva `{ id, nome }`: o `ReferenciaDto`.
- As validações (`[Required]`, `[Range]`) ficam nos DTOs de entrada (slide 38).
- Porquê: slide 33. As linhas /// são para o Swagger (slide 78).

✓ **Ponto de controlo.** compila. Contar: 10 ficheiros na pasta `DTOs`.

# Mappers: copiar campos, e mais nada

Controllers/ | DTOs/ | **Mappers/** | Services/ | Erros/ | Repositories/ | Data/ | Entities/

## Fazer

- Na pasta `Mappers`, três classes, uma por entidade.
- Em cada uma, encontrar os três métodos: `ToDto`, `ToEntity`, `AplicarEm`.

## Ficheiros

+ `Mappers/CategoriaMapper.cs`
+ `Mappers/ModoEntregaMapper.cs`
+ `Mappers/ProdutoMapper.cs`

## Explicar

- **Métodos de extensão**: o `this` no primeiro parâmetro deixa escrever `produto.ToDto()` sem mexer na classe `Produto`.
- `ToDto` → GET, `ToEntity` → POST (entidade nova), `AplicarEm` → PUT (entidade que já existe).
- A imagem passa tal como está (`byte[]`): é o `System.Text.Json` que a converte em Base64, e de volta.
- **Sem regras e sem base de dados**: se precisa de um `if` de negócio, é no Service (slide 37).

✓ **Ponto de controlo.** compila.

# Repositories: o único que fala com a BD

Controllers/ | DTOs/ | Mappers/ | Services/ | Erros/ | **Repositories/** | Data/ | Entities/

## Fazer

- Para cada entidade, primeiro a **interface** (Adicionar > Novo Item > Interface), depois a classe.
- No `ProdutoRepository`, encontrar `AsNoTracking`, `Include` e `SaveChangesAsync`.

## Ficheiros

+ `Repositories/ICategoriaRepository.cs`
+ `Repositories/CategoriaRepository.cs`
+ `Repositories/IModoEntregaRepository.cs`
+ `Repositories/ModoEntregaRepository.cs`
+ `Repositories/IProdutoRepository.cs`
+ `Repositories/ProdutoRepository.cs`

## Explicar

- **Interface = o que se pode pedir; classe = como se faz.** Os Services só conhecem a interface.
- `AsNoTracking()` nas listas: só para ler, mais leve.
- `Include(p => p.categoria)`: traz a categoria no mesmo `SELECT`.
- As gravações devolvem `bool`: se a base de dados recusar, `false`, e não uma exceção.
- Rever: slide 40.

✓ **Ponto de controlo.** compila. Procurar `AppDbContext` no projeto (Ctrl+Shift+F): só aparece em `Data/` e `Repositories/`.

# Erros e Services: as regras do negócio

Controllers/ | DTOs/ | Mappers/ | **Services/** | **Erros/** | Repositories/ | Data/ | Entities/

## Fazer

- Primeiro a pasta `Erros`: `CodigosErro` e `ErrosApi`.
- Depois `Services`: `Resultado`, `Imagens`, e interface + classe para cada entidade.
- No `CategoriaService.DeleteAsync`, encontrar a regra "categoria com produtos".
- **2.d:** compilar.

## Ficheiros

+ `Erros/CodigosErro.cs`
+ `Erros/ErrosApi.cs`
+ `Services/Resultado.cs`
+ `Services/Imagens.cs`
+ `Services/ICategoriaService.cs`
+ `Services/CategoriaService.cs`
+ `Services/IModoEntregaService.cs`
+ `Services/ModoEntregaService.cs`
+ `Services/IProdutoService.cs`
+ `Services/ProdutoService.cs`

## Explicar

- O Service **não sabe de HTTP**: devolve um `Resultado` (`NaoEncontrado`, `Invalido`, `Conflito`) com um código e uma mensagem.
- Erro esperado → `Resultado`; erro inesperado → exceção (slide 46).
- O `ErrosApi` dá a **todos** os erros o mesmo formato (slide 47).
- Regras: nomes únicos, a categoria tem de existir, não se apaga com produtos, a imagem é PNG, JPEG, GIF ou WebP até 2 MB.

✓ **Ponto de controlo. 2.d:** Compilação > Compilar Solução termina com "1 com êxito". Ainda não se executa.

# Ligar à base de dados e registar as peças

## Fazer

- **2.e:** no `appsettings.json`, a `ConnectionStrings` com a `DefaultConnection`.
- **2.f:** substituir o `Program.cs` pelo do guia. As linhas novas vêm destacadas.
- **2.g:** compilar.

## Ficheiros

✎ `appsettings.json` · + DefaultConnection
✎ `Program.cs` · todo

**comparar com**
» `GestaoLoja/appsettings.json`

## Explicar

- **A mesma connection string da Gestão de Loja**: LocalDB, base de dados `FrutariaPWeb`.
- `AddDbContext` + seis `AddScoped(interface, classe)`: ninguém faz `new` (slide 41).
- `AddProblemDetails(ErrosApi.Personalizar)`: liga o formato único dos erros.
- `AddSwaggerGen(...)`: lê o RESTfulAPI.xml com os comentários /// (slide 78).
- A ordem do pipeline conta: `UseExceptionHandler` primeiro, `MapControllers` no fim (slide 42).

✓ **Ponto de controlo. 2.g:** compila com 0 erros e 0 avisos.

# Controllers: os endpoints, finalmente



## Fazer

- Na pasta `Controllers`: primeiro `ApiControllerBase`, depois um controller por recurso.
- Adicionar > **Classe** (mais rápido do que Adicionar > Controlador).
- Num controller, ler um método de cada verbo.
- **2.i:** compilar.

## Ficheiros

+ `Controllers/ApiControllerBase.cs`
+ `Controllers/CategoriasController.cs`
+ `Controllers/ModosEntregaController.cs`
+ `Controllers/ProdutosController.cs`

## Explicar

- `[Route("api/produtos")]` + `[HttpGet("{id:int}")]` = o endpoint (slide 43).
- O mesmo desenho em todos: chamar o Service; se correu mal, `return Erro(r)`; se correu bem, `Ok` (200), `CreatedAtAction` (201) ou `NoContent` (204).
- O `Erro(r)` do `ApiControllerBase` traduz o `Resultado` no status code (slide 44).
- Não há endpoints de imagem: a imagem vai no JSON do `POST` e do `PUT`, em Base64 (slide 36).

✓ **Ponto de controlo. 2.i:** compila com 0 erros e 0 avisos.

# Executar: a API completa no Swagger

## Fazer

- F5 (botão ► https).
- No Swagger: `GET /api/categorias` > Try it out > Execute.
- Ver a janela da consola enquanto o pedido corre.
- Seguir o mesmo pedido no diagrama de sequência do GET.

## Ficheiros

**nada para criar**

» a janela da consola · o SELECT

» a resposta no Swagger · status, headers, body

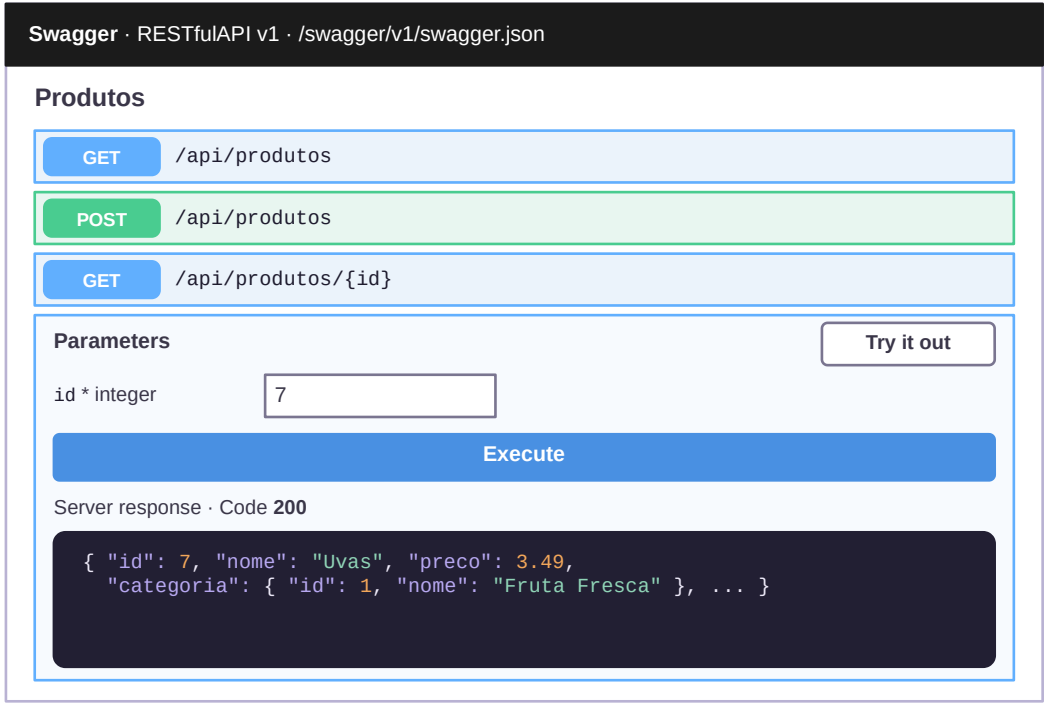
## Explicar

- É a figura 6 da ficha: três grupos, **Categorias**, **ModosEntrega** e **Produtos**.
- Na consola aparece o `SELECT` que o EF Core gerou: o pedido foi mesmo à base de dados.
- Cada camada fez a sua parte: slide 52.
- Experimentar um erro: `GET /api/categorias/999` dá `404` com o DTO de erro.

✔ **Ponto de controlo.** `GET /api/categorias` responde `200` com as categorias da Frutaria, e `GET /api/categorias/999` responde `404` com `"codigo": "CATEGORIA_NAO_ENCONTRADA"`.

ETAPA 2.J · SWAGGER

# Swagger: a ementa, com um botão para experimentar




1. **Abrir.** F5: o browser abre em `https://localhost:<porta>/swagger`.
2. **Escolher.** Clicar num endpoint para o expandir.
3. **Try it out.** Liga os campos: o `id`, ou o JSON do body.
4. **Execute.** Envia o pedido a sério: grava mesmo na BD!
5. **Ler.** O `curl`, o Request URL, o código e o body da resposta.

### OpenAPI e Swagger UI

O OpenAPI é o contrato em JSON (`swagger.json`). O Swagger UI é a página que o desenha. Só existe em Development.

ETAPA 2.J · SWAGGER

# Swagger documentado: comentários /// e exemplos

DTOs/CriarProdutoDto.cs e ProdutosController.cs (excertos)

```
/// <summary>O preço, maior do que zero, com duas
/// casas decimais no máximo.</summary>
/// <example>2.99</example>
[Range(0.01, 99999999.99, ErrorMessage = "...")]
public decimal Preco { get; set; }

/// <summary>Cria um produto.</summary>
/// <response code="201">O produto criado.</response>
/// <response code="409">PRODUTO_NOME_REPETIDO.</response>
[HttpPost]
public async Task<ActionResult<ProdutoDto>> Create(
    CriarProdutoDto dto)
```

O Try it out já traz um body válido: a Papaia, com uma imagem de 16 x 16 em Base64. Antes, vinha `"string"` e `0`.

*O que o Swagger mostra*

**POST /api/produtos** Cria um produto.
`preco*` number · minimum 0.01 · example 2.99
O preço, maior do que zero, com duas casas...
`201` O produto criado. `409` PRODUTO_NOME_REPETIDO.

## Duas peças

`RESTfulAPI.csproj`: `GenerateDocumentationFile` junta os /// num ficheiro `RESTfulAPI.xml`.

`Program.cs`: `IncludeXmlComments(...)` no `AddSwaggerGen` (o Swagger lê-o).

## E no Spring Boot?

`@Operation(summary = ...)` ↔ /// `<summary>` no método.

`@Schema(description, example)` ↔ `<summary>` e `<example>` na propriedade.

`@NotBlank`, `@Size` ↔ `[Required]`, `[StringLength]`: estes já aparecem sozinhos no Swagger.

Em .NET também há atributos (pacote `Swashbuckle.AspNetCore.Annotations`); os /// não precisam de pacote.

# Dev Tunnel: um endereço público para a API

## Fazer

- Seta ▾ do botão ► https > Túneis de Desenvolvimento > **Criar um Túnel...**
- Nome `DevTunnelPWeb`, **Persistente**, **Público**.
- Escolher o túnel no mesmo menu e F5.
- Na página de aviso do Dev Tunnels: Continuar.

## Ficheiros

**nada para criar**

» o URL do browser · ...devtunnels.ms/swagger

**guardar para a app e o Postman**

\+ o URL sem /swagger

## Explicar

- Para o telemóvel, `localhost` é o próprio telemóvel: a app não chega ao nosso PC.
- O túnel é uma ligação **de saída** do PC para a Microsoft, que devolve um URL público.
- **Persistente**: o URL não muda. **Público**: a app não faz login no túnel.
- Os slides seguintes: o esquema, os tipos e acessos, os passos e o endereço.

✔ **Ponto de controlo.** o Swagger abre pelo endereço do túnel, por exemplo `https://abcd1234-7123.uks1.devtunnels.ms/swagger/index.html` (figura 8 da ficha).

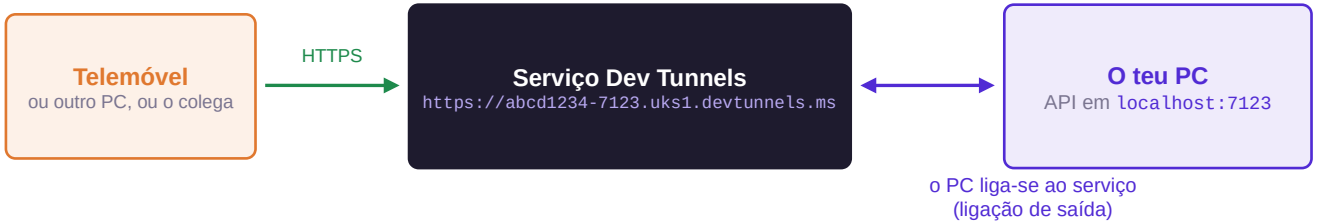
ETAPA 3 · DEV TUNNEL

# Dev Tunnel: o localhost só existe no teu PC

## Sem túnel




## Com Dev Tunnel




### Porquê

Na fase de desenvolvimento, a API não está num servidor. O túnel dá-lhe um endereço público, sem abrir portas no router.

### Quem pode entrar

**Privado**, **Organizacional** ou **Público**. Para as apps: Público (slide seguinte).

### Quanto tempo dura o endereço

**Temporário** ou **Persistente**. Para as apps: Persistente (slide seguinte).

ETAPA 3 · DEV TUNNEL

# Temporário ou persistente, e quem pode entrar

**Tipo de Túnel**: quanto tempo dura o endereço

### Temporário

Um **URL novo** sempre que o Visual Studio arranca. Serve para uma experiência rápida; numa app, obrigava a mudar o endereço todos os dias.

### ✓ Persistente (o da ficha)

O **mesmo URL** em todas as execuções, enquanto o túnel existir. É o que se põe na app e no Postman, uma vez.

**Access**: quem pode usar o endereço

### Privado

Só a **conta que criou** o túnel, depois de fazer login Microsoft no browser. Bom para testar sozinho; uma app recebe a **página de login** em vez de JSON.

### Organizacional

As contas da **mesma organização** (por exemplo, todas as `@isec.pt`), também com login. Para mostrar a colegas no browser; as apps continuam a receber o login.

### ✓ Público (o da ficha)

Quem tiver o URL, **sem login**. O Visual Studio diz que "não é recomendado": para dados de teste numa aula, chega. Numa API a sério, protege-se com o JWT (próxima ficha).

ETAPA 3.A E 3.B · DEV TUNNEL

# Criar e usar o Dev Tunnel no Visual Studio

1. **Abrir o menu.** Na barra de cima, a seta ▾ do botão ► **https** → **Túneis de Desenvolvimento** → **Criar um Túnel...**

2. **Conta.** Escolher a conta Microsoft (ou GitHub). Sem conta não há túnel.

3. **Configurar.** Nome: `DevTunnelPWeb` · Tipo de Túnel: **Persistente** · Access: **Público**. OK.

4. **Ativar.** No mesmo menu, Túneis de Desenvolvimento → escolher `DevTunnelPWeb`.

5. **Executar.** F5. O browser abre `https://abcd1234-7123.uks1.devtunnels.ms/swagger` (figura 8).

6. **Primeira visita.** Aparece uma página de aviso do Dev Tunnels: **Continuar** (Continue). Só no browser.

### O URL para a app

É o do túnel **sem** `/swagger`:
`https://abcd1234-7123.uks1.devtunnels.ms`
Vai para o `baseUrl` da app e do Postman.

### As apps e a página de aviso

Uma app não carrega em Continuar: manda o header `X-Tunnel-Skip-AntiPhishing-Page: true` em cada pedido.

### A API tem de estar a correr

O túnel só encaminha. PC desligado, ou F5 parado: o URL responde erro.

ETAPA 3.B · DEV TUNNEL

# O mesmo Swagger, por outro endereço

**Sem túnel**

```
https://localhost:7123/swagger/index.html
```

**Com o túnel ativo (F5)**

```
https://abcd1234-7123.uks1.devtunnels.ms/swagger/index.html
```




### Request URL e Curl mudam

No Swagger aberto pelo túnel, cada pedido mostra o endereço do túnel. O pedido vai à Microsoft e volta ao vosso PC; a resposta é a mesma.

### Para a app e o Postman

O endereço **sem** `/swagger/index.html`: `https://abcd1234-7123.uks1.devtunnels.ms`. É o `baseUrl`.

### Cada um tem o seu

O id e a porta são de cada aluno; a região é a que o serviço escolher. Copiar sempre o endereço do próprio browser.

# Testar: os sucessos e os erros

## Fazer

- Seguir o roteiro de testes (dois slides à frente), primeiro no **Swagger**.
- Depois no **Postman**: uma coleção, a variável `baseUrl`, um environment local e outro do túnel.
- Imagem (pedidos 8.1 a 8.3): converter num site (guia 3.d) e colar o Base64 no campo "imagem". No Swagger, trocar o "string" do exemplo.
- Provocar os erros: 400, 404, 409, e ler o DTO de erro.

## Ficheiros

**nada para criar**

» a resposta de cada pedido
» o header Location do POST
» o body dos erros · codigo, detail, errors

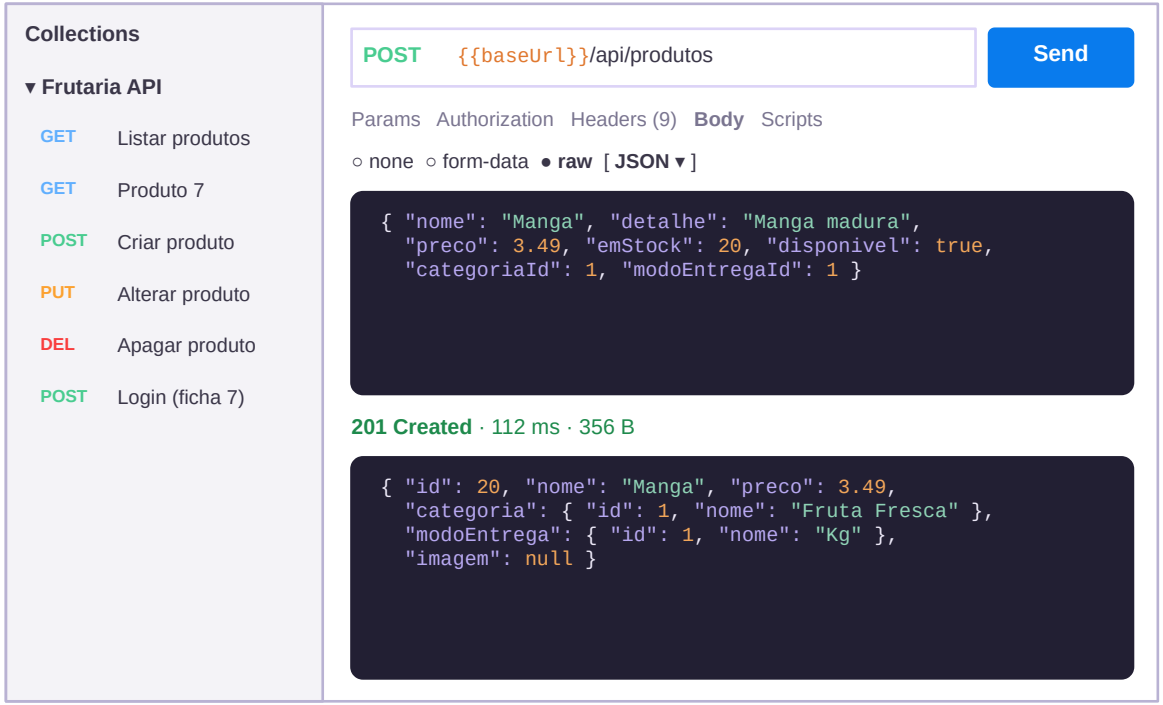
## Explicar

- Um teste tem um **esperado**: se o status não bate, há uma camada a fazer o trabalho errado.
- 201 + `Location`: onde ficou o recurso novo.
- Nos erros, a app mostra o `detail` e decide pelo `codigo` (slide 47).
- Um 409 do princípio ao fim: slide 59.

✓ **Ponto de controlo.** os pedidos do roteiro dão todos o status esperado, e no fim a base de dados fica como estava.

TESTAR · POSTMAN

# Postman: pedidos guardados, variáveis, e coleções




| Environment | baseUrl |
|---|---|
| Local | https://localhost:7123 |
| Dev Tunnel | https://abcd1234-7123.uks1.devtunnels.ms |

### Trocar de servidor

Muda-se o environment, no canto superior direito. Os pedidos ficam iguais.

### Body: raw + JSON

Escolher **JSON** põe o `Content-Type: application/json`. Esquecer dá `415`.

ETAPA 3.C · TESTAR

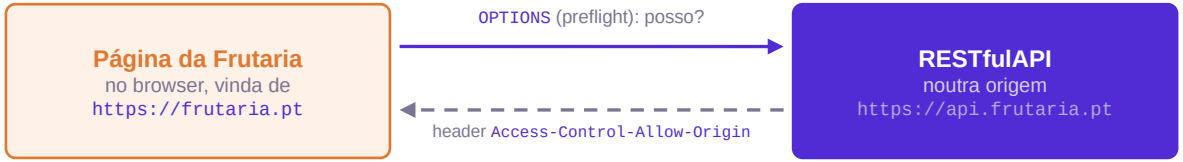
# Roteiro de testes: dez pedidos que provam a API

| # | Verbo | Endpoint | Body | Esperado |
|---|---|---|---|---|
| 1 | GET | `/api/categorias` | | `200`: 4 categorias, pela ordem |
| 2 | GET | `/api/produtos?categoriaId=1` | | `200`: 11 produtos de Fruta Fresca |
| 3 | POST | `/api/produtos` | a Manga do slide anterior | `201` + `Location: /api/produtos/20` |
| 4 | POST | `/api/produtos` | a mesma Manga, outra vez | `409`: já existe um produto "Manga" |
| 5 | POST | `/api/produtos` | com `"preco": 0` | `400`: `errors.preco` |
| 6 | GET | `/api/produtos/20` | | `200`: a Manga, com a categoria |
| 7 | PUT | `/api/produtos/20` | a Manga com `"preco": 3.99` | `204` (e um GET mostra 3.99) |
| 8 | DELETE | `/api/produtos/20` | | `204` |
| 9 | GET | `/api/produtos/20` | | `404` |
| 10 | DELETE | `/api/categorias/1` | | `409`: a categoria tem produtos |

O id 20 é o primeiro livre depois dos 19 produtos de exemplo; se já houver outros, será diferente. Usem o do `Location`.

TESTAR E EXPOR · CORS

# CORS: porque o browser bloqueia o frontend web




## Só nos browsers

O CORS é uma regra do browser, para proteger quem navega. Apps Android, iOS, Windows, o Postman e o curl **não** o aplicam.

## Sem o header

O browser **bloqueia** a resposta. Na consola (F12): `blocked by CORS policy`.

## Origem

Esquema + host + porta. `localhost:5001` e `localhost:7123` já são origens diferentes.

## O Swagger não sofre

A página do Swagger vem da própria API: mesma origem.

Program.cs: permitir uma origem

```
builder.Services.AddCors(o => o.AddDefaultPolicy(p => p
    .WithOrigins("https://localhost:5001")
    .AllowAnyHeader().AllowAnyMethod()));
...
app.UseCors();     // antes do UseAuthorization
```

## Nunca qualquer origem

`AllowAnyOrigin()` só em testes. Em produção, a lista das origens certas.

PRÁTICA · PONTO DE SITUAÇÃO

# A API está pronta se...

- [ ] **Compila sem erros nem avisos**
  Compilação > Compilar Solução
- [ ] **O Swagger mostra os três grupos**
  Categorias, ModosEntrega, Produtos
- [ ] **GET devolve os dados da Frutaria**
  os mesmos que a Gestão de Loja mostra
- [ ] **POST dá 201 e o header Location**
  e o produto aparece num GET a seguir
- [ ] **Validação dá 400 com errors por campo**
  por exemplo `"preco"`, em camelCase
- [ ] **Regras dão 409 com o codigo certo**
  nome repetido, categoria com produtos
- [ ] **Nenhum AppDbContext fora do sítio**
  só em Data, Repositories e Program.cs
- [ ] **O Dev Tunnel abre o Swagger**
  persistente, público, DevTunnelPWeb

# Extra: a API escreve um diário

Controllers/ | DTOs/ | Mappers/ | **Services/** | **Erros/** | **Repositories/** | Data/ | Entities/

## Fazer

- Só depois de a API estar pronta (slide anterior).
- Alterar cinco ficheiros: as linhas novas vêm destacadas no guia.
- F5 e repetir três pedidos do roteiro: um POST, um 409, um 404.

## Ficheiros

- ✎ `Repositories/CategoriaRepository.cs`
- ✎ `Repositories/ModoEntregaRepository.cs`
- ✎ `Repositories/ProdutoRepository.cs`
- ✎ `Services/ProdutoService.cs`
- ✎ `Erros/ErrosApi.cs`

## Explicar

- `ILogger<T>` entra pelo construtor, como os Repositories: injeção de dependências.
- **Níveis**: `Information` para o que muda nos dados, `Warning` quando a BD recusa.
- O `ErrosApi` escreve cada erro com o `traceId`: o mesmo que a app recebe no DTO de erro.
- O slide seguinte mostra o resultado.

✔ **Ponto de controlo.** na consola aparece "Produto N criado", e o `traceId` de uma resposta de erro aparece na linha do log.

EXTRA · LOGGING

# Logging: o diário da API

Repositories/ProdutoRepository.cs (excerto)

```
public class ProdutoRepository(AppDbContext context,
    ILogger<ProdutoRepository> logger) : IProdutoRepository
...
    catch (DbUpdateException ex)
    {
        logger.LogWarning(ex, "A base de dados recusou a gravação.");
        return false;
    }
```

Consola da API

```
info: RESTfulAPI.Services.ProdutoService[0]
      Produto 20 criado: Manga
info: RESTfulAPI.Erros[0]
      Erro 409 PRODUTO_NOME_REPETIDO em POST /api/produtos
      (traceId 0HN7Q3R2K9S1A:00000004): Já existe um produto...
```

| Nível | Quando |
|---|---|
| `Information` | aconteceu algo normal: produto criado |
| `Warning` | correu mal, mas foi tratado |
| `Error` | falhou e não foi tratado (um 500) |

### Sem pacotes

Pede-se um `ILogger<T>` no construtor, como qualquer serviço. O `appsettings.json` escolhe o nível mínimo.

### O traceId liga as duas pontas

O mesmo `traceId` vai no DTO de erro e na linha do log: a app mostra-o, o suporte procura-o na consola.

A SEGUIR · AUTENTICAÇÃO

# Duas perguntas: quem és? e o que podes fazer?

## Autenticação: quem és?

Provar a identidade: email e palavra-passe. Se correr bem, a API dá à app um **token**.

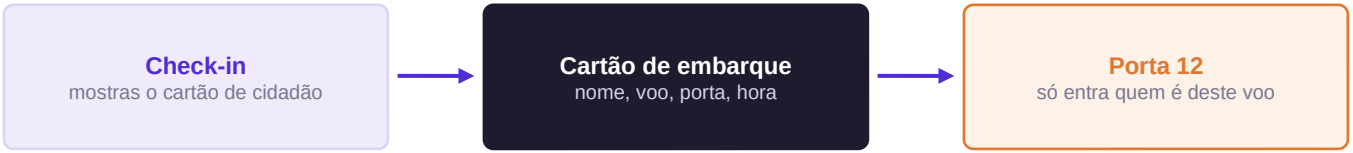
Se falhar: `401 Unauthorized`.

## Autorização: o que podes fazer?

Já se sabe quem és. Agora: o teu role deixa? Um Cliente lê produtos; só o Admin e o Gestor os apagam.

Se falhar: `403 Forbidden`.

**No aeroporto**




**autenticação**
`POST /api/auth/login`

**o token**
que a app guarda

**autorização**
`[Authorize(Roles = "...")]`

### Já viram isto

Na ficha 5: o login, os roles e o "Acesso negado". Lá com um **cookie**.

A SEGUIR · AUTENTICAÇÃO

# Na Gestão de Loja era um cookie; na API é um token

| | Cookie (ficha 5) | Token JWT (API) |
|---|---|---|
| Quem o guarda | o browser, sozinho | a app: código nosso |
| Como viaja | header `Cookie`, enviado **automaticamente** | header `Authorization: Bearer ...`, posto **pela app** |
| Quem o pode usar | browsers, no mesmo site | qualquer cliente: web, Android, iOS, Windows |
| O conteúdo | cifrado: só o servidor o lê | assinado, mas **legível por todos** |
| O servidor lembra-se? | não: o cookie leva tudo | não: o token leva tudo (stateless) |
| Risco típico | CSRF: outro site faz o browser enviá-lo | roubo do token (XSS, telemóvel perdido) |

**Porquê token na API:** uma app Android não é um browser, não gere cookies sozinha, e pode falar com várias APIs em domínios diferentes. O token é texto: qualquer linguagem o guarda e o põe num header. E o formato JWT é um padrão aberto (RFC 7519).

A SEGUIR · O TOKEN

# Um token é a pulseira de um festival




| No festival | Na API | Onde |
|---|---|---|
| a bilheteira | o endpoint de login | `POST /api/auth/login` |
| a pulseira | o token JWT | guardado na app |
| o nome e a zona VIP | as claims: email, role | dentro do token |
| "válida até domingo" | a validade | a claim `exp` |
| o selo que não se imita | a assinatura | a 3.ª parte do token |
| o segurança | o middleware JwtBearer | em cada pedido |

### Sem telefonar

O segurança decide **só** com a pulseira: rápido, e a bilheteira pode estar fechada.

### Pulseira falsa ou de ontem

Selo errado, ou fora de prazo: não entra. Na API: `401`.

A SEGUIR · O TOKEN

# Bearer: quem tem o token, entra

```
Em cada pedido a um endpoint protegido, a app acrescenta este header
GET /api/conta/perfil HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJiN2UyYzFmMC00YTFkIiwiZW1haWwi...
```

**Bearer** quer dizer **portador**, como num cheque ao portador: quem o apresentar, recebe. A API não sabe se quem manda o token é a Ana ou alguém que o copiou.

### Sempre HTTPS

Em HTTP, o token viaja à vista: quem estiver na mesma rede copia-o.

### Validade curta

Um token roubado só serve até expirar. Minutos ou horas, não meses.

### Nunca no URL nem em logs

`?token=...` fica no histórico do browser e nos logs dos servidores.

### Logout = esquecer

A app apaga o token. A API não o "cancela": é válido até expirar.

A SEGUIR · JWT

# JWT: três partes separadas por pontos

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJiN2UyYzFmMC00YTFkIiwiZW1haWwiOiJhbmFAbWFpbC5wdCIsIm5hbWUiOiJBbmEgU2lsdmEiLCJyb2xlIjoiQ2xpZW50ZSIsImlzcyI6IlJFU1RmdWxBUEkiLCJhdWQiOiJGcnV0YXJpYUFwcHMiLCJpYXQiOjE3OTM3MDcyMDAsImV4cCI6MTc5MzcxMDgwMH0.hZVkjS4mExLlkUkuQPvuHoFTFpCwEkVhUu5zagOQiQY
```

**Header**: o tipo e o algoritmo

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Cada parte é JSON em **Base64Url**: só letras, números, - e _, para poder ir num header.

**Payload**: as claims: quem é, até quando

```
{
  "sub": "b7e2c1f0-4a1d",
  "email": "ana@mail.pt",
  "name": "Ana Silva",
  "role": "Cliente",
  "iss": "RESTfulAPI",
  "aud": "FrutariaApps",
  "iat": 1793707200,
  "exp": 1793710800
}
```

**Signature**: o selo

```
HMACSHA256(
  header + "." + payload,
  chave
)
```

Experimentem colar o token em **jwt.io**: o header e o payload leem-se logo.

A SEGUIR · JWT

# O payload: as claims

| Claim | Nome | No exemplo | Para quê |
|---|---|---|---|
| `sub` | subject | `"b7e2c1f0-4a1d"` | o Id do utilizador (AspNetUsers) |
| `email` | email | `"ana@mail.pt"` | mostrar, identificar |
| `name` | nome | `"Ana Silva"` | "Olá, Ana" na app |
| `role` | role | `"Cliente"` | o `[Authorize(Roles = ...)]` |
| `iss` | issuer | `"RESTfulAPI"` | quem emitiu |
| `aud` | audience | `"FrutariaApps"` | para quem foi emitido |
| `iat` | issued at | `1793707200` | quando foi emitido |
| `exp` | expiration | `1793710800` | quando deixa de valer |

### As datas

Segundos desde 1/1/1970 (Unix). `1793707200` é 3/11/2026, 12:00 UTC; o `exp` é uma hora depois.

### Registadas e nossas

`sub`, `iss`, `aud`, `exp`... são padrão (RFC 7519). Podemos juntar as nossas.

### No controller

`User.FindFirstValue(...)`, `User.IsInRole("Admin")`: o .NET lê as claims do token.

A SEGUIR · JWT

# Base64Url não é encriptação: todos leem




Codificar não é esconder: não há chave. O caminho de volta é igual para toda a gente.

## Nunca no token

- A palavra-passe (nem o hash).
- NIF, morada, telemóvel.
- Dados de pagamento.
- Qualquer coisa que não possa aparecer num ecrã alheio.

## Pode ir

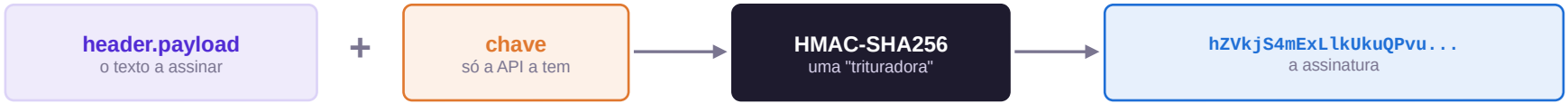
- O Id do utilizador.
- O email e o nome.
- Os roles.
- As datas e o emissor.

**Um JWT é como um postal com um selo de lacre:** qualquer pessoa lê o que lá está escrito. O lacre não esconde nada, mas mostra se alguém mexeu. É o que vem a seguir: a assinatura.

A SEGUIR · JWT

# A assinatura: o selo que denuncia alterações

## Como a API assina (no login)




## A Ana tenta passar a Admin

1. Copia o token, decodifica o payload e troca `"role": "Cliente"` por `"role": "Admin"`.
2. Volta a codificar: o payload muda de `eyJyb2xlIjoiQ2xpZW50ZSJ9` para `eyJyb2xlIjoiQWRtaW4ifQ`.
3. Não tem a chave: deixa a assinatura antiga, ou inventa outra.
4. A API recalcula a assinatura com a **sua** chave: não bate certo. `401`.

### Mudar 1 letra muda tudo

O HMAC dá uma assinatura completamente diferente se mudar um só carácter do texto.

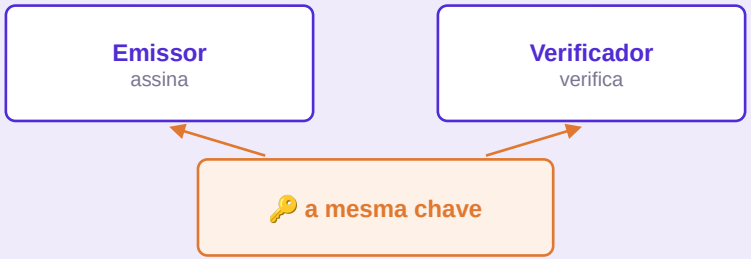
### Sem a chave, não há selo

Calcular a assinatura certa sem a chave é impossível na prática.

A SEGUIR · CHAVES

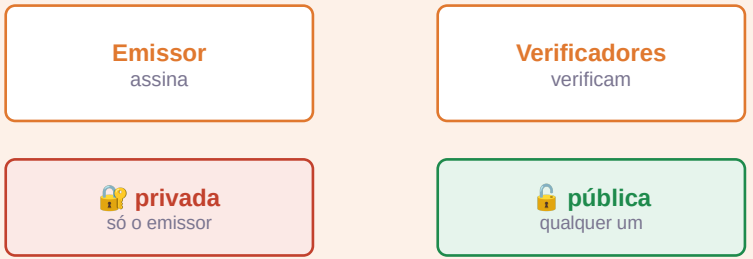
# Uma chave secreta, ou um par: privada e pública

## Simétrica: HS256




Quem verifica também consegue **emitir**. Serve quando emissor e verificador são o mesmo programa: **a nossa API**.

## Assimétrica: RS256, ES256




A pública **só verifica**: não serve para assinar. Pode ser dada a toda a gente. Serve quando quem emite e quem verifica são **programas diferentes**.

Em ambos os casos o token é igual por fora: muda o `alg` do header e a forma de calcular a assinatura.

A SEGUIR · CHAVES

# Chave privada e chave pública: o papel de cada uma

| | 🔐 Chave privada | 🔓 Chave pública |
|---|---|---|
| Quem a tem | só o emissor (o servidor de login) | todas as APIs que aceitam os tokens |
| Para que serve | **assinar**: criar tokens | **verificar** assinaturas |
| Cria tokens? | **sim** | **não** |
| Se for roubada | qualquer um emite tokens de Admin: trocar o par **já** | nada: já era pública |
| Onde fica | cofre (Key Vault, user-secrets); **nunca no git** | publicada, ex.: `/.well-known/jwks.json` |

A regra de ouro, nos dois casos: **o segredo que assina nunca sai do servidor que emite os tokens.**

### A assinatura no banco

Só tu sabes fazer a tua assinatura (a privada). O banco tem a ficha com a tua assinatura, para comparar (a pública). Comparar não chega para a imitar.

### E com HS256?

A chave secreta tem os dois papéis, e por isso trata-se **como uma privada**: cofre, nunca no git, mínimo 32 bytes.

A SEGUIR · EMISSOR

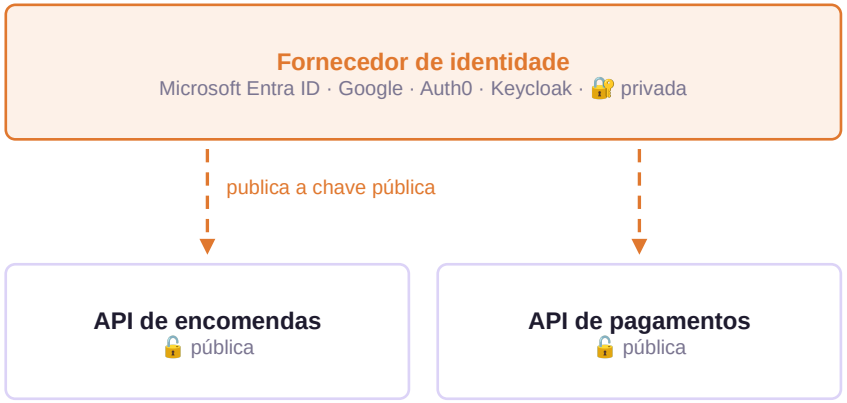
# Quem emite o token, e quem o aceita

## Na nossa API: tudo no mesmo programa




`iss` = "RESTfulAPI" · `aud` = "FrutariaApps"

## Em sistemas maiores: um fornecedor de identidade

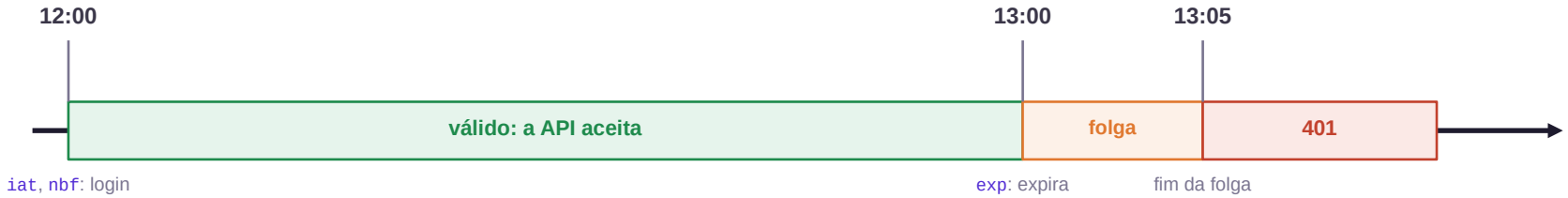



O "Entrar com a Google" é isto: a Google emite, a app aceita.

**iss** (issuer): quem emitiu. **aud** (audience): para quem. A API recusa tokens de outro emissor, ou feitos para outra audiência.

A SEGUIR · VALIDADE

# Validade: um token não dura para sempre




## Porquê curto

Um token roubado não se pode cancelar. Quanto mais curto, menor o estrago.

## Quanto tempo

**Access token**: 15 a 60 minutos. Numa app de aulas, 1 hora chega.

## ClockSkew

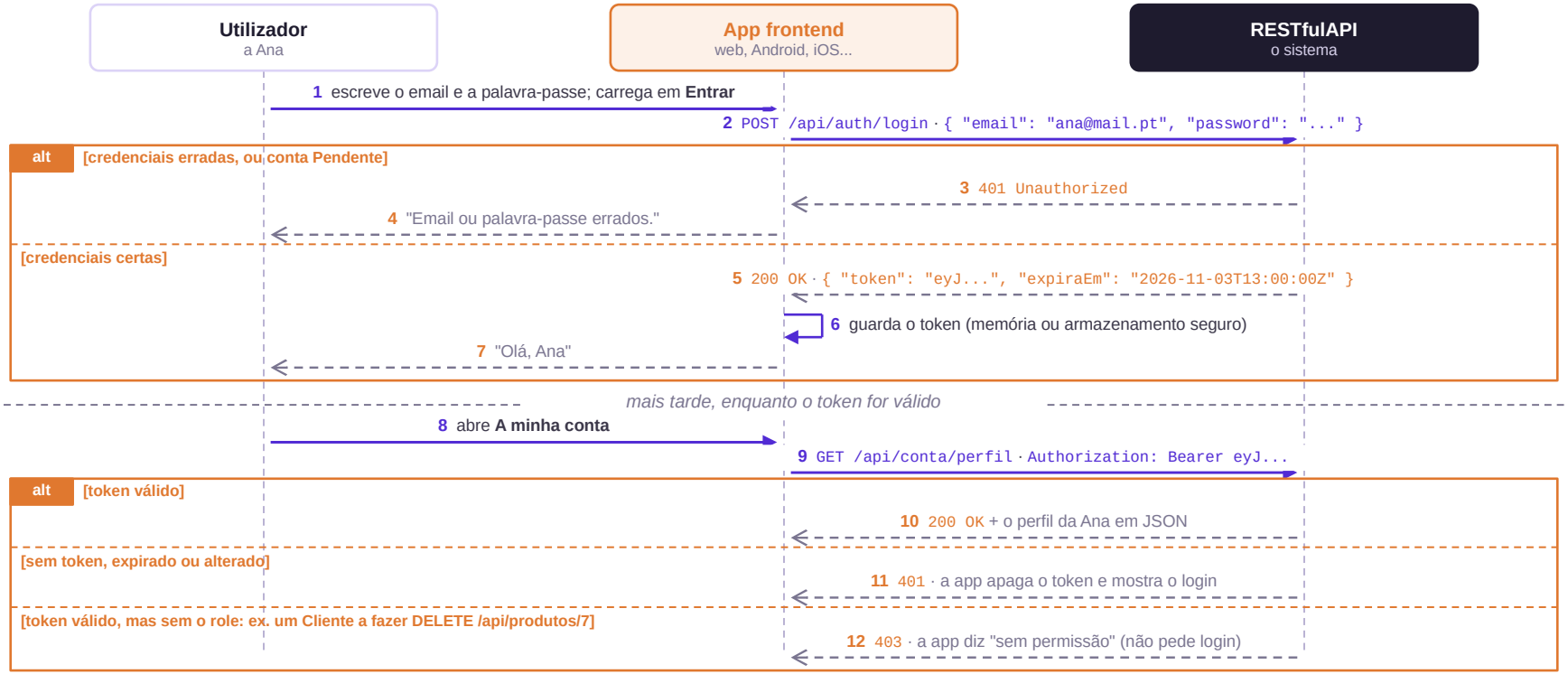
O .NET aceita **5 minutos** de folga (relógios desacertados). Nos testes de expiração: `TimeSpan.Zero`.

### E depois de expirar?

A API responde `401`. A app pede login outra vez, ou usa um **refresh token**: um segundo token, de longa duração e guardado com mais cuidado, que só serve para pedir um access token novo (`POST /api/auth/refresh`). Fica para mais tarde.

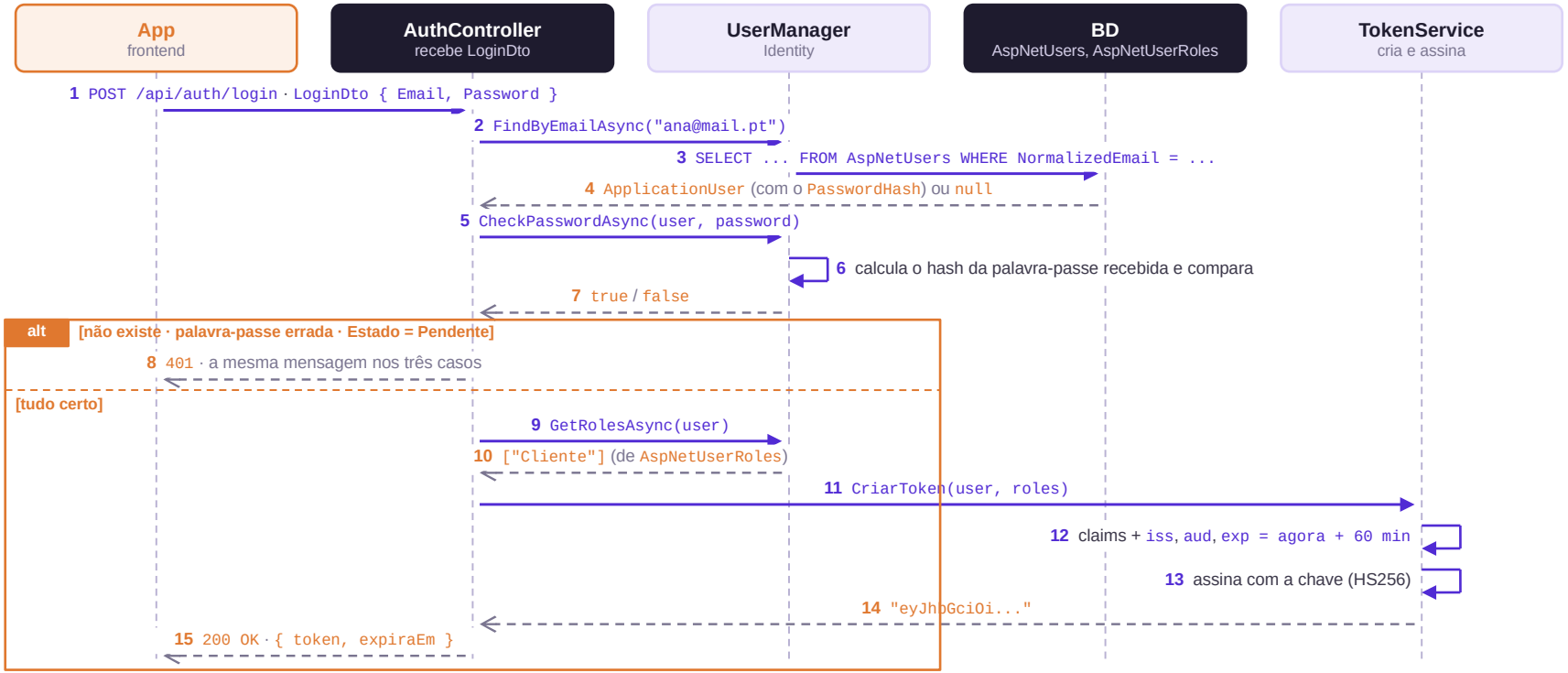
A SEGUIR · DIAGRAMA DE SEQUÊNCIA DE SISTEMA

# Login, guardar o token, e usar o token



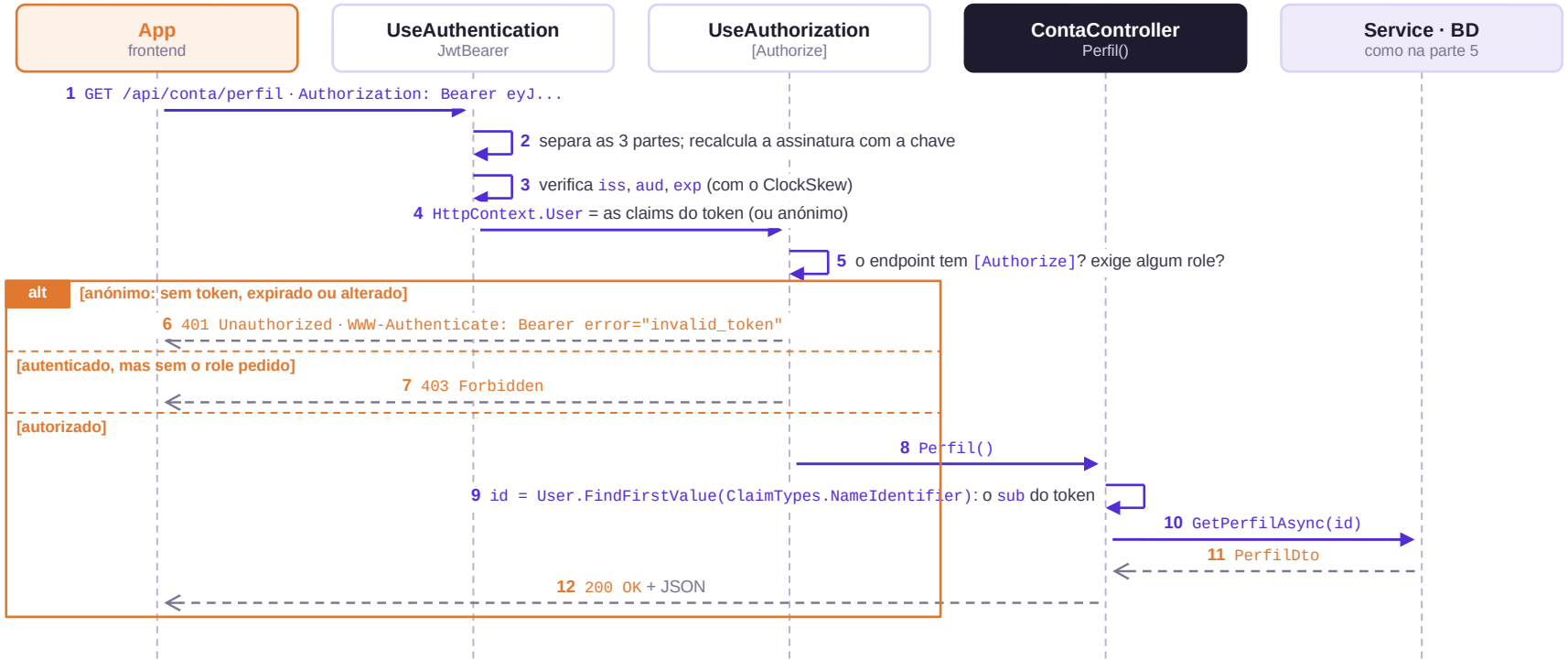
A SEGUIR · POR DENTRO DO LOGIN

# POST /api/auth/login: como nasce o token



A SEGUIR · POR DENTRO DE UM PEDIDO PROTEGIDO

# GET /api/conta/perfil: o token em ação



A SEGUIR · VALIDAÇÃO DO TOKEN

# O que a API verifica em cada token

**Formato**
3 partes, Base64Url

→

**Algoritmo**
o esperado (HS256); recusa `none`

→

**Assinatura**
recalculada com a chave

→

**Emissor**
`iss` = "RESTfulAPI"

→

**Audiência**
`aud` = "FrutariaApps"

→

**Validade**
`nbf` ≤ agora ≤ `exp` + folga

Todas certas: `User` fica com as claims. Uma errada: o utilizador fica anónimo, e o `[Authorize]` responde `401`.

Program.cs (próxima ficha): as mesmas verificações, em código

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o => o.TokenValidationParameters = new()
    {
        ValidateIssuer = true,   ValidIssuer = "RESTfulAPI",
        ValidateAudience = true, ValidAudience = "FrutariaApps",
        ValidateLifetime = true, ClockSkew = TimeSpan.FromMinutes(1),
        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    });
```

## O pacote

`Microsoft.AspNetCore.Authentication.JwtBearer`, versão **8.0.x** (a do .NET 8).

## A chave fora do código

`Jwt:Key` vem da configuração: em desenvolvimento, dos **user-secrets**. No git, nunca.

A SEGUIR · NA APP

# Onde a app frontend guarda o token

| Onde | Sobrevive a fechar? | O JavaScript lê? | Risco | Quando usar |
|---|---|---|---|---|
| variável em memória | não | sim, só a app | perde-se ao recarregar | web: o mais simples e seguro |
| `sessionStorage` | até fechar o separador | sim | XSS lê-o | web: aceitável, token curto |
| `localStorage` | sim, para sempre | sim | XSS lê-o; fica lá | **evitar** |
| cookie `HttpOnly`, `Secure`, `SameSite` | sim | **não** | CSRF (o `SameSite` trava) | web: o mais seguro (o servidor gere) |
| `SecureStorage` (.NET MAUI), Keychain (iOS), Keystore (Android) | sim | · | cifrado pelo sistema | **apps nativas: sempre** |

### XSS: o perigo nos browsers

Um script malicioso injetado na página lê tudo o que o JavaScript lê, incluindo o `localStorage`.

### Nunca

No código-fonte, no URL, em logs, num ficheiro de texto, nas preferências normais da app.

A SEGUIR · NA APP

# A app e o token: quatro momentos

1

### Depois do login

Guardar o `token` e o `expiraEm`.

2

### Em cada pedido

Pôr o header `Authorization: Bearer <token>`.

3

### Resposta 401

Apagar o token e voltar ao ecrã de login.

4

### Logout

Apagar o token. Não é preciso chamar a API.

C# (MAUI, Blazor, consola): o header num HttpClient

```
var http = new HttpClient
    { BaseAddress = new Uri(baseUrl) };
http.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", token);

var perfil = await http
    .GetFromJsonAsync<PerfilDto>("api/conta/perfil");
```

JavaScript (página web): o header num fetch

```
const r = await fetch(baseUrl + "/api/conta/perfil", {
  headers: {
    "Authorization": "Bearer " + token,
    "Accept": "application/json"
  }
});
if (r.status === 401) { /* voltar ao login */ }
```

A SEGUIR · TESTAR

# Testar com token: Swagger, Postman e jwt.io

## Swagger

1. Executar o `POST /api/auth/login`.
2. Copiar o `token` da resposta.
3. Botão **Authorize** (o cadeado), colar o token.
4. Os pedidos seguintes levam o header.

Precisa de `AddSecurityDefinition` no `AddSwaggerGen` (próxima ficha).

## Postman

1. Pedido de login na coleção.
2. No separador **Scripts** (post-response) guardar o token numa variável.
3. Na coleção: **Authorization** → Bearer Token → `{{token}}`.
4. Todos os pedidos herdam.

## jwt.io

Colar o token para ver o header e o payload, e confirmar o `role` e o `exp`.

Só com tokens de teste: um token real colado num site é um token entregue.

Postman · pedido de login · Scripts → Post-response

```
const r = pm.response.json();
pm.collectionVariables.set("token", r.token);   // os outros pedidos usam {{token}}
```

A SEGUIR · PRÓXIMA FICHA

# O que a API vai ganhar na próxima ficha

| | |
|---|---|
| **Pacotes NuGet** | `Microsoft.AspNetCore.Authentication.JwtBearer` 8.0.x; o Identity já lá está (`AppDbContext`). |
| **appsettings** | Secção `Jwt`: `Issuer`, `Audience`, `DurationMinutes`. A `Key` nos user-secrets. |
| **Program.cs** | `AddIdentityCore<ApplicationUser>()`, `AddAuthentication().AddJwtBearer(...)`, `UseAuthentication()`. |
| **DTOs** | `LoginDto`, `RegistoDto`, `TokenDto { Token, ExpiraEm }`, `PerfilDto`. |
| **Services** | `TokenService` (cria e assina), `AuthService` (login, registo com `Estado = Pendente`). |
| **Controllers** | `AuthController` (`login`, `registar`), `ContaController` (`perfil`). |
| **Proteger** | `[Authorize(Roles = "Admin,Gestor")]` nos POST, PUT e DELETE; os GET ficam abertos. |

QUANDO ALGO FALHA

# Erros frequentes

| Sintoma | Causa e solução |
|---|---|
| Não há pasta `Controllers`; o `Program.cs` tem `app.MapGet` | Faltou **Usar controles** ao criar o projeto. Criar de novo. |
| `Unable to resolve service ...`, ou a API nem arranca | Falta um `AddScoped` (de um Service ou de um Repository) no `Program.cs`. |
| `415 Unsupported Media Type` | Body sem `Content-Type: application/json` (Postman: raw + JSON). |
| `400 JSON_INVALIDO` | O body está vazio, ou o JSON está mal escrito (vírgula a mais, `1,49`). Com "3.49" entre aspas, o 400 é DADOS_INVALIDOS, em errors.preco. |
| `404` num endpoint que existe | Rota errada (`/api/produto`), ou `{id:int}` com texto no URL. |
| A API não arranca: `crit: ... ligar à base de dados` | Connection string diferente da da Gestão de Loja, ou a base de dados não existe. |
| `JsonException: A possible object cycle was detected` | Um controller a devolver entidades. Devolver DTOs. |
| `500` e `NullReferenceException` no `ProdutoMapper` | Falta o `Include(p => p.categoria)` no `ProdutoRepository`. |
| Postman: `SSL Error: unable to verify the first certificate` | Certificado de desenvolvimento: desligar a verificação SSL só para localhost. |
| Dev Tunnel devolve uma página de login, não JSON | Túnel com acesso **Privado**. Criar com **Público**. |
| Browser: `blocked by CORS policy` | Falta `AddCors` + `UseCors` com a origem do frontend. |

PARA LEVAR DA AULA

# Do pedido HTTP à base de dados, e de volta

### REST e recursos

Endereços que identificam coisas, verbos que dizem o que fazer, e nenhum pedido depende do anterior.

### HTTP

Pedido: verbo, caminho, headers, body. Resposta: status code, headers, body. 2xx, 4xx, 5xx.

### JSON e DTOs

O objeto vira texto e volta. As entidades ficam na cozinha; viajam DTOs, um por operação.

### Camadas

Controller (HTTP), Service (regras), Repository (dados), Mapper (conversão). Só para baixo, por interfaces.

### Testar e expor

Swagger e Postman para testar; Dev Tunnel para um endereço público; CORS para o browser.

### Token JWT

Header, payload, assinatura. Legível por todos, impossível de alterar sem a chave. Curto, e bem guardado.